

Roteiro de condução — Encontro 4

Aula de 24/09/2026 • Containers, Docker e Compose • 4 horas

Professor: Arimatéia Júnior • Desenvolvimento de Software Integrado — DevOps • UNIFOR

Roteiro separado da apresentação. Abrange a capa e os 78 slides de conteúdo já existentes, com fala sugerida, condução, demonstrações, respostas e transições. A apresentação não foi alterada.

Use o PDF para consulta durante a aula e o DOCX para anotações pessoais. As falas são sugestões: adapte o vocabulário à turma sem ler o roteiro inteiro em voz alta.

Preparação e uso rápido

Navegação pelos blocos

Preparação e uso rápido.....	2
1. Abertura e direção da aula.....	5
2. Fundamentos: construir o modelo mental.....	7
3. Primeiros containers e primeira verificação.....	15
4. Construção da imagem da API.....	19
5. Integração, rede, dados e saúde.....	25
6. Diagnóstico guiado e uso crítico de IA.....	35
7. Trabalho das equipes no projeto.....	38
8. Fechamento da trilha principal.....	43
9. Slides de consulta: usar conforme a dúvida.....	44
Apoio durante a aula: dúvidas e contingências.....	49
Materiais de apoio e referências.....	51

Objetivo da aula: ao final, cada equipe deve conseguir explicar os componentes de execução, empacotar sua aplicação, integrar as dependências pertinentes e mostrar uma operação e uma falha diagnosticada com evidências. A API de visitas é o exemplo do professor; o projeto dos alunos continua sendo o já escolhido.

Como localizar: E4·NN é o identificador no rodapé. No PDF independente, a página é NN + 1; no conjunto principal, a posição é 107 + NN. Capa: página 1 do independente e posição 107 do conjunto. Use a apresentação completa do encontro; o guia em PowerPoint contém uma seleção e pode omitir alguns IDs.

Horário: os tempos abaixo são relativos ao início, pois o horário de início não foi informado. Anote antes de começar: início ____; intervalo no minuto 95 ____; retorno no minuto 110 ____; equipes no minuto 165 ____; encerramento no minuto 240 ____.

Leia primeiro, se estiver com pouco tempo: este plano, E4·10 (runtimes), E4·32 (API isolada), E4·39–44 (rede/dados/saúde), E4·51 (persistência), E4·58 (falha e recuperação) e o bloco de acompanhamento das equipes.

Antes de os alunos entrarem

1. Abra a apresentação independente, este roteiro e o repositório. Deixe Dockerfile, compose.yaml, server.js e os dois READMEs localizados.
2. Prepare dois terminais na pasta Práticas/encontro-4-containers. A demonstração de API isolada ocupa um terminal em primeiro plano; o outro executa os testes.
3. Confirme Engine ativo, Compose disponível e portas 18080, 18081 e 8080 livres. Use uma cópia conhecida do laboratório. Se houver conflito, identifique o serviço; não interrompa trabalho de terceiros.

4. Baixe as imagens e construa a API antes da aula para reduzir a dependência da internet. Faça o ensaio seguindo o README, anote o total real e encerre a stack normalmente preservando o volume.

5. Aumente a fonte do terminal e teste a projeção. Separe saídas de um ensaio como contingência, identificadas com data; não as apresente como execução ao vivo.

6. Organize as equipes já existentes e confirme o acesso ao projeto de cada uma. As instruções de terminal são Bash/zsh; em Windows, use o ambiente preparado ou adapte para PowerShell/curl.exe.

Preparação no terminal, a partir da raiz do repositório:

```
cd 'Práticas/encontro-4-containers'
docker version
docker compose version
docker ps -a
docker pull node:22-bookworm-slim
docker pull postgres:16-alpine
docker build -t e4-api:local .
docker compose config --quiet
```

Esses comandos preparam e verificam o ambiente; os testes funcionais completos estão nos roteiros do laboratório. Antes de E4-19, mantenha os nomes e portas da demonstração livres. Não use limpeza global nem remoção de volumes para preparar a aula.

Plano de quatro horas

Minutos	Duração	Slides	Condução
0-10	10 min	Capa, E4-01-02	Apresentar resultado e percurso.
10-50	40 min	E4-03-18	Construir os conceitos antes dos comandos.
50-65	15 min	E4-19-24	Primeiros containers e gabarito.
65-95	30 min	E4-25-35	Imagem da API, build, execução isolada e gabarito.
95-110	15 min	Intervalo	Retornar com compose.yaml aberto.
110-145	35 min	E4-36-53	Integração, teste, persistência e gabarito.
145-165	20 min	E4-54-59	Dois casos comentados e uma falha ao vivo.
165-235	70 min	E4-60-67	Trabalho no projeto com orientação e revisão embutidas.
235-240	5 min	E4-68-69	Evidências e conexão com o encontro 5.
Consulta	Fora das 4h	E4-70-78	Usar por necessidade, sem nova obrigação.

Os tempos por slide são referências de ritmo, não cronômetros rígidos. A capa usa um minuto do primeiro bloco. Os 70 minutos das equipes incluem orientações breves e feedback enquanto trabalham; não acrescente uma exposição de oito slides antes de começar a atividade.

Dinâmica pedagógica

Em cada bloco: apresente o problema → defina os componentes → percorra o exemplo → observe o resultado → peça justificativa → revele a resposta. Evite cobrar a definição de um termo que ainda não explicou.

Nas quatro verificações: ouça respostas curtas, procure a justificativa e corrija com o slide seguinte. Os gabaritos estão reproduzidos neste roteiro para sua consulta; não os leia antes de dar espaço à turma. São verificações formativas em grupo, sem introduzir nota individual.

No terminal: antes do Enter, diga a intenção; depois da execução, aponte a parte da saída que a sustenta. Não descreva todo comando como “agora funcionou”. Diga se comprovou execução, saúde, escrita/leitura ou persistência.

Se atrasar: reduza a leitura de tabelas e trate E4·42, 47 e 48 como panoramas. Comente E4·56–57 sem editar o YAML e mantenha E4·58 como única falha ao vivo. Preserve a base dos runtimes, imagem/container, rede, persistência, gabaritos e o bloco das equipes. Os slides de consulta não entram na exposição obrigatória.

1. Abertura e direção da aula

Janela: 0–10 min.

Capa — Encontro 4

Referência: página 1 do PDF independente; posição 107 do conjunto principal. **Tempo:** 1 minuto.

Fala sugerida: “Hoje vamos entender como o nosso programa vira uma aplicação executável em um ambiente organizado, e como demonstrar que os serviços realmente conversam. Vamos do conceito à prática no projeto de vocês.”

Condução: acolha a turma, situe a continuidade do curso e apresente o título. Evite abrir com um quiz sobre Docker. Avance para o objetivo antes de detalhar comandos.

E4.01 — Do programa ao ambiente integrado

Localização: PDF independente, página 2; apresentação principal, posição 108. **Tempo sugerido:** 5 min.

O que o aluno deve levar: Resultado: outra pessoa consegue subir o projeto, testar seu funcionamento e entender sua configuração.

Fala de abertura sugerida: “Hoje vamos transformar o projeto de vocês em algo que outra pessoa consiga executar. Primeiro vamos entender as peças, depois observar uma aplicação e, por fim, aplicar esse raciocínio ao projeto da equipe.”

Como conduzir: Aponte entender, observar e aplicar nessa ordem. Relacione com o repositório que os grupos já têm, sem pedir definições técnicas neste início.

Explicação para desenvolver: Apresente a aula como uma construção de conceitos. Primeiro os alunos precisam reconhecer o que está sendo executado e qual ferramenta assume cada responsabilidade. Só depois entram os comandos, a integração e as questões. O exemplo do professor será uma API de visitas; cada equipe continuará a aplicação escolhida nos encontros anteriores. Não comece pedindo que a turma explique Docker ou descubra uma configuração que ainda não foi ensinada.

Transição: “Para chegar a esse resultado, vamos dividir as quatro horas em etapas.”

E4.02 — Roteiro das quatro horas

Localização: PDF independente, página 3; apresentação principal, posição 109. **Tempo sugerido:** 4 min.

O que o aluno deve levar: As verificações vêm depois da explicação; as equipes aplicam os conceitos ao próprio projeto.

Fala de abertura sugerida: “Vocês terão tempo de trabalho no próprio projeto. A explicação prepara esse trabalho; a API de visitas será apenas o nosso exemplo comum.”

Como conduzir: Informe o intervalo e o início da prática. Avise que haverá perguntas depois das explicações, com correção em sala. Registre no quadro o horário real correspondente ao minuto 165.

Explicação para desenvolver: O cronograma soma 240 minutos, incluindo 15 minutos de intervalo e 70 minutos de prática no projeto. Os gabaritos pertencem ao tempo de cada bloco,

não são uma segunda aula. Faça uma previsão breve antes de mostrar cada resposta. Há material de consulta após o fechamento para dúvidas sobre digest, multi-stage e verificação em CI. Preserve o laboratório; se a turma avançar devagar, explique as distinções essenciais e deixe exemplos de consulta para estudo posterior.

Transição: “Antes de falar em Docker, precisamos identificar o que uma aplicação usa para funcionar.”

2. Fundamentos: construir o modelo mental

Janela: 10–50 min.

E4-03 — Uma aplicação precisa de mais que o código

Localização: PDF independente, página 4; apresentação principal, posição 110. **Tempo sugerido:** 3 min.

O que o aluno deve levar: A execução depende da combinação código + runtime + dependências + configuração + serviços.

Fala de abertura sugerida: “O código é uma parte do sistema. Este arquivo precisa de um runtime, usa bibliotecas, recebe configuração e conversa com outro serviço. Levar somente o arquivo para outra máquina pode deixar essas peças faltando.”

Como conduzir: Abra server.js e package.json, apenas localizando seus papéis. Aponte a conexão externa; não explique ainda todas as linhas da API.

Explicação para desenvolver: Comece pelo problema conhecido: ter o arquivo server.js não significa ter uma aplicação executável. Mostre no repositório o arquivo, o package.json e a variável de conexão. Explique biblioteca como código que a aplicação utiliza, runtime como o ambiente que executa seu programa e banco como outro serviço. Nesta etapa basta reconhecer essas partes; porta, volume e rede terão explicação própria. Evite apresentar container como uma solução que automaticamente corrige código ou credenciais.

Transição: “Vamos separar primeiro o arquivo que guardamos da execução que acontece na máquina.”

E4-04 — Programa é arquivo; processo é execução

Localização: PDF independente, página 5; apresentação principal, posição 111. **Tempo sugerido:** 3 min.

O que o aluno deve levar: Um container em execução terá um processo principal; quando esse processo termina, o container para.

Fala de abertura sugerida: “Uma partitura no papel não produz som sozinha. Com o programa ocorre algo parecido: o arquivo fica armazenado, e sua execução cria um processo consumindo recursos.”

Como conduzir: Percorra o desenho arquivo → comando → processo. Diga que duas execuções podem ser independentes. Encerre a analogia lembrando que processo é uma entidade do sistema operacional.

Explicação para desenvolver: Use uma analogia simples: a partitura guardada não produz som; alguém precisa executá-la. O arquivo de programa permanece no disco, enquanto o processo usa CPU, memória e descritores de arquivo. Iniciar o mesmo programa duas vezes pode criar dois processos. Quando o processo termina, o programa continua existindo como arquivo. No nosso exemplo, node server.js cria um processo Node que atende requisições. Um container não muda o fato de que, ao final, existem processos executando.

Transição: “Quem oferece memória, CPU e acesso a arquivos para esse processo?”

E4-05 — Sistema operacional: recursos para os processos

Localização: PDF independente, página 6; apresentação principal, posição 112. **Tempo sugerido:** 2 min.

O que o aluno deve levar: Compartilhar o kernel não significa compartilhar todos os arquivos, processos visíveis ou endereços de rede.

Fala de abertura sugerida: “O kernel é a parte do sistema operacional que administra recursos para os processos. Uma distribuição Linux também inclui programas e bibliotecas; essas duas coisas não são sinônimos.”

Como conduzir: Desenhe uma linha “kernel” sob dois processos. Use esse desenho como referência quando chegar a containers e VMs. Não aprofunde chamadas de sistema.

Explicação para desenvolver: Separe kernel de distribuição Linux: uma distribuição também reúne bibliotecas, programas e configuração. Uma imagem Debian ou Alpine pode trazer arquivos de uma distribuição sem iniciar um kernel próprio. Isso prepara a comparação com VMs, que virá depois. Não é necessário detalhar chamadas de sistema; os alunos precisam compreender que aplicações usam recursos gerenciados por uma camada comum. O escopo da aula são containers Linux.

Transição: “Acima dessa base, algumas aplicações também precisam de um runtime de linguagem.”

E4-06 — Runtime da aplicação: quem executa o programa

Localização: PDF independente, página 7; apresentação principal, posição 113. **Tempo sugerido:** 3 min.

O que o aluno deve levar: No laboratório, Node.js é o runtime da API; pg é uma biblioteca; PostgreSQL é outro serviço.

Fala de abertura sugerida: “No nosso exemplo, Node executa JavaScript no servidor. Em outra equipe, essa responsabilidade pode ser de Python ou de uma JVM. A versão desse componente faz parte do ambiente da aplicação.”

Como conduzir: Escolha a linha da tabela correspondente à stack predominante da turma e outra para comparação. Para Go, resalte que distribuir um binário não elimina necessidades do sistema.

Explicação para desenvolver: Explique o termo antes de usá-lo em comandos. JavaScript no servidor pode usar Node.js, código Python depende do interpretador e de suas bibliotecas, e bytecode Java usa uma JVM. O runtime da linguagem normalmente é instalado dentro da imagem junto com a aplicação. Uma aplicação Go compilada pode distribuir um binário sem instalar uma VM de linguagem separada; isso não elimina suas necessidades de sistema. A versão do runtime influencia APIs e comportamento, por isso ela faz parte do contrato de execução.

Transição: “Agora conseguimos definir o que fica dentro do ambiente de um container.”

E4-07 — Container: processo em um ambiente isolado

Localização: PDF independente, página 8; apresentação principal, posição 114. **Tempo sugerido:** 3 min.

O que o aluno deve levar: Empacotar a aplicação e executá-la em um container reduz diferenças de ambiente; não elimina todas elas.

Fala de abertura sugerida: “Na execução de um container há processos usando um ambiente com isolamento configurado. A imagem fornece arquivos, e a instância recebe configuração e estado próprios. Um container parado ainda pode existir como objeto do Docker.”

Como conduzir: Aponte processo, isolamento e compartilhamento do kernel. Evite começar pela expressão “VM leve”, pois ela esconde justamente a diferença que será ensinada.

Explicação para desenvolver: Diga explicitamente: container não é só uma pasta compactada, nem uma máquina virtual pequena. Na execução, o runtime prepara um ambiente a partir de uma imagem e inicia processos com isolamento configurado. A imagem fornece arquivos; a instância acrescenta configuração e estado. O container pode estar parado e continuar existindo como objeto gerenciado. Ao falar que compartilha kernel, refira-se ao host Linux em que executa, que também pode ser uma VM.

Transição: “Esse isolamento aparece de maneiras concretas quando olhamos rede, arquivos e recursos.”

E4-08 — Isolamento: o que fica separado

Localização: PDF independente, página 9; apresentação principal, posição 115. **Tempo sugerido:** 2 min.

O que o aluno deve levar: Isolamento, limites e persistência são escolhas configuradas; o nome “container” não define sozinho essas políticas.

Fala de abertura sugerida: “Cada processo pode ter uma visão delimitada de recursos. Por isso duas aplicações podem usar arquivos diferentes e enxergar redes diferentes mesmo no mesmo ambiente Linux.”

Como conduzir: Relacione filesystem a dependências, rede a endereços e limites a consumo. Diga que montagens e privilégios alteram esse alcance; isolamento não é uma garantia absoluta de segurança.

Explicação para desenvolver: Relacione cada mecanismo a um efeito observável, sem transformar a aula em administração avançada do kernel. A separação de rede explica por que localhost da API não é o banco. Limites de recursos explicam por que um processo pode ter memória restrita. A visão de filesystem explica por que dependências de duas imagens não precisam ser instaladas no mesmo diretório do host. O isolamento é configurável; montagens e privilégios podem ampliar o acesso. Não apresente container como uma fronteira de segurança absoluta.

Transição: “Precisamos distinguir a origem desses arquivos da instância que os utiliza.”

E4-09 — Imagem e container são objetos diferentes

Localização: PDF independente, página 10; apresentação principal, posição 116. **Tempo sugerido:** 3 min.

O que o aluno deve levar: Um novo container recebe a imagem e a configuração escolhidas; não herda automaticamente edições de outra instância.

Fala de abertura sugerida: “Pense em um molde e em duas peças produzidas com ele. A imagem é a origem; cada container tem identidade e estado próprios. A analogia ajuda, mas a imagem também contém metadados de execução.”

Como conduzir: Aponte uma imagem e duas instâncias no desenho. Explique que alterar um arquivo em uma instância não reescreve a imagem original nem modifica automaticamente a outra.

Explicação para desenvolver: A imagem é um artefato de arquivos e metadados que serve de origem. Um container é a instância criada a partir dela, com um comando, variáveis, rede e estado gravável. A mesma imagem pode criar várias instâncias. Alterar um arquivo dentro de uma delas não altera a imagem original. O nome de uma imagem pode ser uma referência mutável, assunto para a consulta sobre tags e digest. A distinção principal é artefato versus execução, sem exigir ainda que o aluno conheça registry.

Transição: “Quem prepara esse ambiente para o processo começar a executar?”

E4.10 — Runtime de containers: quem prepara o isolamento

Localização: PDF independente, página 11; apresentação principal, posição 117. **Tempo sugerido:** 3 min.

O que o aluno deve levar: Node.js e um runtime de containers não se substituem: atuam em camadas diferentes.

Fala de abertura sugerida: “A palavra runtime está aparecendo em dois níveis. Node executa JavaScript. O runtime de containers prepara e inicia processos com isolamento. Um não substitui o outro.”

Como conduzir: Percorra o desenho de fora para dentro. Mencione containerd e runc como exemplos da arquitetura típica, sem pedir que a turma memorize esses nomes. Volte a Node no fim.

Explicação para desenvolver: Este é o ponto que costuma gerar confusão. Node executa o código JavaScript; um runtime de containers prepara e inicia processos sob regras de isolamento. Em uma arquitetura Linux típica do Docker, containerd gerencia o ciclo de vida e um runtime de baixo nível, como runc, cria o processo conforme a configuração OCI. OCI é um conjunto de especificações de interoperabilidade, não outra aplicação que os alunos precisam instalar. O desenho simplifica detalhes como shims: o objetivo é distinguir responsabilidades, não decorar todos os processos internos.

Transição: “O Docker organiza uma experiência de uso sobre esses componentes.”

E4.11 — Docker: ferramentas para trabalhar com containers

Localização: PDF independente, página 12; apresentação principal, posição 118. **Tempo sugerido:** 2 min.

O que o aluno deve levar: Docker é a ferramenta de trabalho; imagem é o artefato; container é a instância executável.

Fala de abertura sugerida: “Quando usamos Docker, temos ferramentas para construir imagens e operar containers, redes e armazenamento. O comando no terminal é o cliente dessa operação.”

Como conduzir: Aponte CLI, Engine e Desktop como responsabilidades distintas. Use a instalação presente na sala como exemplo, sem tratar toda instalação como idêntica.

Explicação para desenvolver: Agora associe a tecnologia à ferramenta. Container é um conceito de execução e isolamento; Docker fornece uma experiência para construir e operar esses objetos. Digitar docker não significa que o processo da aplicação foi iniciado dentro do cliente do terminal: há uma API e um serviço atendendo às operações. O cliente pode inclusive controlar um Engine remoto. Docker Desktop reúne componentes e interface para desenvolvimento; não deve ser confundido com o formato da imagem nem com o runtime da linguagem.

Transição: “Vamos acompanhar o caminho de um comando concreto.”

E4.12 — O caminho de um docker run

Localização: PDF independente, página 13; apresentação principal, posição 119. **Tempo sugerido:** 3 min.

O que o aluno deve levar: O terminal controla a execução; o registry distribui a imagem; os processos executam no host do Engine.

Fala de abertura sugerida: “Ao executar docker run, o cliente pede ao Engine a criação e a partida de uma instância. A imagem pode precisar ser obtida primeiro; depois o processo é iniciado no ambiente preparado.”

Como conduzir: Siga as setas na ordem. Pare no registry e diga que ele armazena imagens, não executa a aplicação. Diferencie esse fluxo de construir uma imagem.

Explicação para desenvolver: Percorra o desenho da esquerda para a direita. A CLI pede ao Engine para criar e iniciar uma instância. Se necessário e conforme a política de obtenção, uma imagem é baixada do registry. O Engine coordena preparação de filesystem, rede e execução usando os componentes de runtime. O processo principal começa com o comando escolhido. A aplicação não roda dentro do registry. Build é outro fluxo: transforma instruções e contexto em uma imagem, não inicia automaticamente um servidor permanente.

Transição: “Precisamos localizar também o ambiente Linux que oferece o kernel.”

E4.13 — Engine e Desktop: onde o container roda

Localização: PDF independente, página 14; apresentação principal, posição 120. **Tempo sugerido:** 2 min.

O que o aluno deve levar: VMs e containers podem coexistir: uma VM Linux pode hospedar vários containers.

Fala de abertura sugerida: “Se vocês usam Docker Desktop em um sistema que precisa de uma VM Linux para estes containers, é o kernel desse ambiente Linux que será compartilhado. A imagem não inicia seu próprio kernel.”

Como conduzir: Mostre a posição da VM no esquema mental já construído. No Windows, mantenha a explicação no modo de containers Linux usado nesta aula.

Explicação para desenvolver: Use o ambiente efetivo da turma como exemplo, sem prometer uma arquitetura única para todos os sistemas. Docker Desktop em macOS usa uma VM Linux; no Windows há opções de backend e esta aula assume o modo de containers Linux. Mesmo que o aluno esteja no macOS, uma imagem Debian não inicializa sozinha um kernel Debian. A VM

fornece o ambiente Linux necessário. Containers Windows existem, mas suas regras de compatibilidade estão fora do laboratório.

Transição: “Com uma aplicação e um banco, precisamos descrever mais de um serviço de forma coordenada.”

E4.14 — Docker Compose: descrever a aplicação integrada

Localização: PDF independente, página 15; apresentação principal, posição 121. **Tempo sugerido:** 3 min.

O que o aluno deve levar: Dockerfile descreve como construir uma imagem; compose.yaml descreve como executar e integrar os serviços.

Fala de abertura sugerida: “Compose descreve como os serviços de uma aplicação devem ser executados juntos. Aqui teremos app e db. Ele usa o Engine para operar esses recursos; não executa JavaScript no lugar do Node.”

Como conduzir: Aponte os dois serviços e a relação entre eles. Defina serviço como declaração e container como instância. Deixe a leitura de YAML para o bloco de integração.

Explicação para desenvolver: Apresente Compose antes de abrir YAML. Ele organiza a configuração de uma aplicação com um ou mais serviços e usa o Docker Engine para criar e operar os recursos. No laboratório há um serviço app e um serviço db. Serviço é a definição; container é uma instância dessa definição. Compose não executa JavaScript nem instala o banco dentro da API. O arquivo também pode declarar redes, volumes, variáveis e condições de inicialização. Um comando como up aplica essa descrição; isso não substitui testes funcionais.

Transição: “Agora temos base para comparar a estrutura de uma VM com a de containers.”

E4.15 — VM e container: compare a estrutura

Localização: PDF independente, página 16; apresentação principal, posição 122. **Tempo sugerido:** 2 min.

O que o aluno deve levar: VM virtualiza uma máquina; container organiza processos sob isolamento do sistema operacional.

Fala de abertura sugerida: “Na VM, cada sistema convidado tem seu kernel. Neste desenho de containers Linux, as aplicações compartilham o kernel do host Linux, mantendo seus arquivos e configurações delimitados.”

Como conduzir: Compare os desenhos de baixo para cima. Marque a presença de kernels no lado das VMs e a camada compartilhada no lado dos containers.

Explicação para desenvolver: Na VM, o hypervisor apresenta hardware virtual e cada convidado tem seu sistema operacional e kernel. Nos containers Linux, processos e bibliotecas de várias aplicações compartilham o kernel do host Linux. Isso pode reduzir a necessidade de manter um sistema operacional convidado por aplicação, mas não autoriza prometer tempos fixos de startup ou superioridade universal. O desenho é simplificado; o host dos containers também pode ser uma VM, como já vimos no Desktop.

Transição: “Essa diferença estrutural orienta escolhas, mas não dá uma resposta universal para todo sistema.”

E4-16 — VM e container: diferenças que orientam a escolha

Localização: PDF independente, página 17; apresentação principal, posição 123. **Tempo sugerido:** 2 min.

O que o aluno deve levar: Containerizar um monólito não o transforma em microsserviços: arquitetura e empacotamento são decisões diferentes.

Fala de abertura sugerida: “Se preciso de outro sistema operacional ou de condições específicas de kernel, uma VM pode ser necessária. Se preciso separar versões e dependências de aplicações compatíveis, imagens diferentes podem atender bem.”

Como conduzir: Explore dois casos da tabela, sem ler todas as células. Acrescente que VMs podem hospedar containers: as tecnologias também podem ser combinadas.

Explicação para desenvolver: Explore um exemplo de cada escolha. Uma aplicação que depende de outro sistema operacional ou de um kernel específico pode exigir VM. APIs com bibliotecas e versões de linguagem diferentes podem ser empacotadas em imagens separadas, executando no mesmo ambiente Linux compatível. Na nuvem é comum usar VMs como hosts de containers. Tempo de partida depende tanto da ferramenta quanto da aplicação, do download e dos dados inicializados; não use números fixos como propriedade garantida.

Transição: “Vamos fechar a base dizendo com precisão o que a containerização resolve.”

E4-17 — O que a containerização resolve — e seus limites

Localização: PDF independente, página 18; apresentação principal, posição 124. **Tempo sugerido:** 2 min.

O que o aluno deve levar: A execução confiável combina artefato conhecido, configuração correta, recursos adequados e verificação.

Fala de abertura sugerida: “Empacotar dependências reduz diferenças de ambiente, mas uma configuração errada continua errada. Também precisamos pensar em dados, acesso, recursos e disponibilidade.”

Como conduzir: Escolha os exemplos senha errada e volume sem backup. Relacione cada limite a uma responsabilidade da equipe; não trate o uso de Docker como solução automática.

Explicação para desenvolver: Feche a base conceitual explicando o alcance real. A mesma imagem reduz variações de arquivos e dependências, mas uma senha errada continua errada. Um volume local não protege de perda de disco. Um container sem limites definidos pode consumir recursos em excesso. Compose atende bem ao escopo de um host e pode ser usado em cenários compatíveis com isso; não oferece sozinho um cluster distribuído com alta disponibilidade.

Transição: “Os recursos seguintes fazem sentido porque atendem a essas necessidades.”

E4-18 — Mapa dos recursos do Docker

Localização: PDF independente, página 19; apresentação principal, posição 125. **Tempo sugerido:** 2 min.

O que o aluno deve levar: Escolha o recurso pelo problema: imagem não guarda o banco; volume não publica uma porta.

Fala de abertura sugerida: “Este mapa serve para localizar ferramentas, não para decorar uma lista de comandos. Imagens tratam o artefato; rede liga serviços; armazenamento preserva estado; inspeção ajuda a investigar.”

Como conduzir: Faça uma passagem breve pelas famílias. Avise que construção avançada e outros recursos estarão nos slides de consulta. Não abra demonstrações extras aqui.

Explicação para desenvolver: Apresente o mapa como orientação para o restante da aula. Os recursos serão retomados quando houver um problema concreto: dados ao integrar o banco, logs ao diagnosticar, limites ao discutir consumo. Docker também oferece recursos de construção avançada e integração com ferramentas de desenvolvimento; eles estão na consulta. Não é preciso memorizar todas as opções da CLI para entender as responsabilidades. A equipe deve justificar os recursos que realmente utiliza.

Transição: “Vamos observar primeiro um container que executa uma tarefa curta.”

3. Primeiros containers e primeira verificação

Janela: 50–65 min.

E4-19 — Primeira demonstração: um comando que termina

Localização: PDF independente, página 20; apresentação principal, posição 126. **Tempo sugerido:** 2 min.

O que o aluno deve levar: O container não precisa permanecer ativo: sua duração acompanha o processo principal.

Fala de abertura sugerida: “Neste comando, a tarefa é imprimir a versão do Node e terminar. O Docker cria a instância, executa o processo e, com --rm, remove a instância ao final.”

Como conduzir: Execute `docker version` e depois o comando do slide. Aponte v22.x na saída. Diga que a imagem permanece e que a instalação de Node do host não foi alterada.

Explicação para desenvolver: Execute este comando primeiro, sem banco nem Dockerfile. A saída deve ser uma versão 22.x, mas o patch depende da imagem obtida pela tag. A instalação de Node no host não é substituída por esse comando. O processo principal é curto: imprime e sai. O --rm remove o container depois da saída, não a imagem baixada. Se houver falha antes da execução, distinga daemon indisponível de falha de obtenção da imagem. Esta é uma observação guiada, não uma pergunta antes da definição.

Comando ou trecho de apoio: leia a intenção antes de executar. Trechos de configuração podem ser parciais; o arquivo completo do laboratório é a referência.

```
docker version
docker run --rm node:22-bookworm-slim node --version
```

O que observar e dizer sobre a saída: Saída esperada: versão do cliente e do servidor em docker version; depois v22.x.x. O patch pode variar. O comando termina e devolve o prompt. Se não houver acesso ao Engine, ainda não chegamos à execução do Node.

Transição: “Uma aplicação servidora tem um comportamento diferente: precisa continuar atendendo.”

E4-20 — Segunda demonstração: um servidor que permanece ativo

Localização: PDF independente, página 21; apresentação principal, posição 127. **Tempo sugerido:** 4 min.

O que o aluno deve levar: A porta publicada é uma entrada para chegar ao processo; ela não cria um servidor por si só.

Fala de abertura sugerida: “Agora o processo cria um servidor HTTP e permanece em execução. A porta publicada no host leva a requisição até a porta em que esse processo escuta no container.”

Como conduzir: Copie o comando completo do roteiro de primeiros passos. Mostre “Olá, turma!” pelo curl ou navegador. Aponte 18080 no cliente e 3000 no processo.

Explicação para desenvolver: Antes de executar, confirme que não existe uma instância e4-primeiro de uma demonstração anterior e que a porta 18080 está livre. Se já existir uma instância sua, use o roteiro de parada e remoção adiante; não interrompa serviços de outras tarefas. O

código inline usa somente a biblioteca padrão de Node. O bind 0.0.0.0 aceita tráfego na interface do container; a publicação no host fica limitada ao endereço local. A sintaxe multilinha foi preparada para Bash/zsh. Mostre a resposta no terminal ou navegador.

Comando ou trecho de apoio: leia a intenção antes de executar. Trechos de configuração podem ser parciais; o arquivo completo do laboratório é a referência.

```
docker run -d --name e4-primeiro \
  -p 127.0.0.1:18080:3000 \
  node:22-bookworm-slim node -e '
    require("node:http").createServer((req, res) => {
      res.end("Olá, turma!\n");
    }).listen(3000, "0.0.0.0");
  '
curl --retry 5 --retry-connrefused --retry-delay 1 \
  -fsS http://localhost:18080
```

O que observar e dizer sobre a saída: Saída esperada: um ID de container e, após o curl, "Olá, turma!". O -d deixa o terminal livre. Se o primeiro acesso for cedo demais, o curl do roteiro repete a tentativa. O processo precisa estar escutando, além da publicação existir.

Transição: "Com o servidor ativo, podemos consultar o que existe e o que está acontecendo."

E4-21 — Observar e executar comandos em uma instância

Localização: PDF independente, página 22; apresentação principal, posição 128. **Tempo sugerido:** 3 min.

O que o aluno deve levar: logs mostra o que foi emitido; inspect mostra configuração/estado; exec executa; stats mede consumo.

Fala de abertura sugerida: "Cada comando responde a uma pergunta: quais instâncias estão ativas, quais processos existem, o que foi registrado e quanto recurso está sendo usado."

Como conduzir: Demonstre ps, top e exec; use inspect para um campo específico. Se logs estiver vazio, explique que esse servidor não emite logs de requisição.

Explicação para desenvolver: Execute antes da parada e remoção. exec não cria uma nova instância da imagem: inicia um comando dentro do container existente, que precisa estar ativo. inspect consulta o objeto gerenciado e funciona também para uma instância parada; selecione apenas o campo necessário. O servidor inicial não emite logs de requisições, portanto logs vazio não é falha do Docker. stats ajuda a observar consumo, mas não substitui histórico e alertas. Comandos de inspeção não corrigem o problema por si só.

O que observar e dizer sobre a saída: ps: instância ativa; top: processo Node; exec: versão do Node; inspect: running; stats: uma amostra de consumo. logs vazio é possível neste servidor porque ele não registra requisições.

Transição: "Observar, parar e remover são operações diferentes no ciclo de vida."

E4-22 — Ciclo de vida: criar, observar, parar e remover

Localização: PDF independente, página 23; apresentação principal, posição 129. **Tempo sugerido:** 2 min.

O que o aluno deve levar: Imagem, instância e processo têm ciclos de vida relacionados, mas não são o mesmo objeto.

Fala de abertura sugerida: "Stop encerra a execução, mas mantém a instância. Start retoma essa instância. Run criaria outra. Remover a instância não significa remover sua imagem."

Como conduzir: Pare e retome apenas e4-primeiro, repetindo o acesso. Ao concluir, pare e remova apenas essa demonstração. Aponte a diferença entre ps e ps -a.

Explicação para desenvolver: Leia cada comando junto de seu efeito. ps lista containers em execução e ps -a inclui os parados. stop solicita encerramento e start retoma a mesma instância. rm remove a instância parada; os arquivos da imagem continuam disponíveis. docker top mostra os processos da instância. Não confunda start com run: start retoma uma instância existente, run cria uma nova. Ao terminar a demonstração, pare e remova apenas e4-primeiro. Isso não deve afetar a stack do laboratório nem outros serviços do host.

O que observar e dizer sobre a saída: Após stop, ps -a ainda mostra a instância; após start, ela volta a atender; após rm, a instância deixa de existir. A imagem permanece. Só opere sobre o nome e4-primeiro escolhido nesta demonstração.

Transição: “Já temos conceitos e observações suficientes para a primeira verificação.”

E4-23 — Verificação | Componentes e responsabilidades

Localização: PDF independente, página 24; apresentação principal, posição 130. **Tempo sugerido:** 2 min.

O que o aluno deve levar: Discuta primeiro. O próximo slide traz as respostas comentadas.

Fala de abertura sugerida: “Agora quero ouvir as justificativas de vocês usando o desenho e o que acabamos de executar. Respondam primeiro em dupla, sem pesquisar comandos novos.”

Como conduzir: Dê cerca de 45 segundos para discussão e peça respostas curtas de pares diferentes. Ouça as três antes de abrir o gabarito; anote a confusão mais frequente.

Perguntas e respostas esperadas — consulta do professor; revelar no slide seguinte:

1. **Pergunta:** Em node server.js dentro de um container, qual é o papel de Node e qual é o papel do runtime de containers?

Resposta esperada: Node executa o JavaScript. O runtime de containers prepara o isolamento e inicia o processo nesse ambiente. São camadas diferentes.

2. **Pergunta:** Duas instâncias criadas da mesma imagem são o mesmo container?

Resposta esperada: Não. Compartilham a origem, mas possuem identidade, configuração e estado de instância próprios.

3. **Pergunta:** Compose instala um kernel e executa o JavaScript da aplicação?

Resposta esperada: Não. Compose declara e coordena serviços usando o Engine. O processo usa seu runtime de linguagem e o kernel do host Linux.

Transição: “Vamos comparar as respostas com as responsabilidades de cada componente.”

E4-24 — Respostas comentadas | Componentes e responsabilidades

Localização: PDF independente, página 25; apresentação principal, posição 131. **Tempo sugerido:** 2 min.

O que o aluno deve levar: Uma resposta completa identifica o componente envolvido e explica a evidência.

Fala de abertura sugerida: “O ponto central é separar responsabilidades. O runtime da linguagem executa o código; o de containers prepara a execução isolada; Compose coordena serviços.”

Como conduzir: Leia cada resposta junto da pergunta correspondente. Se alguém chamou duas instâncias de “o mesmo container”, volte rapidamente ao desenho de E4-09.

Correção comentada e retomada se houver erro:

1. **Retome a pergunta:** Em node server.js dentro de um container, qual é o papel de Node e qual é o papel do runtime de containers?

Explique: Node executa o JavaScript. O runtime de containers prepara o isolamento e inicia o processo nesse ambiente. São camadas diferentes.

Se a turma não entendeu: Se misturarem os runtimes, desenhe duas caixas: “executar JavaScript” e “preparar isolamento”. Peça que coloquem Node na primeira.

2. **Retome a pergunta:** Duas instâncias criadas da mesma imagem são o mesmo container?

Explique: Não. Compartilham a origem, mas possuem identidade, configuração e estado de instância próprios.

Se a turma não entendeu: Se disserem que duas instâncias são a mesma coisa, pergunte qual delas teria o arquivo alterado. Retome identidade e estado próprios.

3. **Retome a pergunta:** Compose instala um kernel e executa o JavaScript da aplicação?

Explique: Não. Compose declara e coordena serviços usando o Engine. O processo usa seu runtime de linguagem e o kernel do host Linux.

Se a turma não entendeu: Se atribuírem kernel ou JavaScript ao Compose, volte à seta Compose → Engine e ao processo Node.

Transição: “Vamos passar do servidor de demonstração para uma aplicação com dependência externa.”

4. Construção da imagem da API

Janela: 65–95 min.

E4.25 — Agora vamos empacotar uma aplicação real

Localização: PDF independente, página 26; apresentação principal, posição 132. **Tempo sugerido:** 2 min.

O que o aluno deve levar: A aplicação de exemplo ensina o padrão; a equipe deve adaptar esse padrão à aplicação que já escolheu.

Fala de abertura sugerida: “Nossa API agora vai ler e incrementar um contador guardado no PostgreSQL. A biblioteca de acesso ao banco fica na API; o servidor do banco é outro serviço.”

Como conduzir: Mostre GET como leitura e POST como incremento. Avise que o número inicial depende do estado do volume. Relacione essa separação ao projeto das equipes.

Explicação para desenvolver: A demonstração anterior separou container de integração. Agora acrescente uma dependência de verdade. Mostre que o serviço do banco é outro processo, não uma biblioteca instalada dentro de Node. O comando POST executa uma atualização atômica no banco e GET lê o valor. A demonstração é fornecida para o professor; não obriga as equipes a abandonar seu projeto. As credenciais aula/aula são fictícias e servem apenas à stack local de laboratório.

Transição: “Antes de construir, vamos localizar o papel de cada arquivo.”

E4.26 — Arquivos do laboratório: uma responsabilidade por arquivo

Localização: PDF independente, página 27; apresentação principal, posição 133. **Tempo sugerido:** 2 min.

O que o aluno deve levar: Para reproduzir a execução, o repositório precisa trazer todos os arquivos necessários, além das instruções.

Fala de abertura sugerida: “Cada arquivo responde a uma necessidade: código, dependências, construção, integração, inicialização ou instruções de uso. Encontrar esses papéis ajuda a investigar erros depois.”

Como conduzir: Abra a pasta do laboratório e localize os arquivos, sem percorrer todo o código. Mostre package-lock.json antes de mencionar npm ci.

Explicação para desenvolver: Abra a pasta e localize cada arquivo antes do build. O package-lock.json faz parte do exemplo; não solicite npm ci sem fornecer um lockfile. O Dockerfile e o Compose têm papéis diferentes, que ficarão claros nos passos seguintes. O init.sql inicializa o banco quando seu diretório de dados está vazio; ele não é um sistema de migrações. O README concentra os comandos completos, enquanto alguns slides dividem trechos longos para facilitar a leitura.

Transição: “O Dockerfile define a construção da imagem da aplicação.”

E4.27 — Dockerfile: a receita de construção da imagem

Localização: PDF independente, página 28; apresentação principal, posição 134. **Tempo sugerido:** 3 min.

O que o aluno deve levar: Build produz a imagem. A API começa a atender somente quando um container executa o comando definido.

Fala de abertura sugerida: “Este arquivo reúne as instruções que produzem a imagem. Parte delas prepara arquivos e dependências; outra parte configura como uma futura instância vai iniciar.”

Como conduzir: Mostre o Dockerfile completo. Separe visualmente FROM até COPY das configurações de execução. Explique o conjunto antes de detalhar cada instrução.

Explicação para desenvolver: Apresente o Dockerfile completo e explique que o builder lê instruções para produzir uma imagem. Ele não é um script que deve instalar tudo novamente em cada partida da API. FROM node:22-bookworm-slim fornece Node e bibliotecas do espaço de usuário; não inicia uma VM Debian. As próximas lâminas detalham os dois momentos para evitar a memorização sem entendimento. A escolha de slim é adequada ao exemplo; uma imagem menor não é automaticamente compatível ou segura.

Comando ou trecho de apoio: leia a intenção antes de executar. Trechos de configuração podem ser parciais; o arquivo completo do laboratório é a referência.

```
FROM node:22-bookworm-slim
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev
COPY --chown=node:node server.js ./
ENV NODE_ENV=production
USER node
EXPOSE 3000
CMD ["node", "server.js"]
```

Transição: “Primeiro vamos olhar o que acontece durante a construção.”

E4-28 — Dockerfile: o que ocorre durante o build

Localização: PDF independente, página 29; apresentação principal, posição 135. **Tempo sugerido:** 3 min.

O que o aluno deve levar: O código e as dependências precisam entrar na imagem pelo build, não por edição manual de uma instância.

Fala de abertura sugerida: “Durante o build, definimos a base e o diretório, copiamos os manifests, instalamos dependências e depois copiamos o código da aplicação.”

Como conduzir: Leia a ordem no arquivo aberto. Relacione cada instrução a seu efeito concreto. Se perguntarem sobre camadas, diferencie alteração de arquivos e metadados.

Explicação para desenvolver: Percorra a tabela com o Dockerfile aberto ao lado. WORKDIR define o diretório das instruções seguintes. A primeira cópia coloca manifests no contexto da instalação. npm ci usa o lockfile e falha se estiver incoerente com package.json. A segunda cópia transfere somente server.js, evitando incluir arquivos locais desnecessários. O chown ajusta propriedade do arquivo copiado. Não diga que toda instrução adiciona uma camada de filesystem: instruções também podem definir metadados.

Transição: “Depois de construir, ainda é necessário definir o processo que vai iniciar.”

E4-29 — Dockerfile: como a instância vai iniciar

Localização: PDF independente, página 30; apresentação principal, posição 136. **Tempo sugerido:** 3 min.

O que o aluno deve levar: RUN executa uma etapa de construção; CMD define o comando padrão para a execução do container.

Fala de abertura sugerida: “CMD fornece o comando padrão da instância. RUN executa uma etapa do build. EXPOSE documenta uma porta; a publicação para o host será definida ao executar.”

Como conduzir: Aponte CMD, USER e EXPOSE. Compare “instalar dependências” com “iniciar o servidor”. Evite dizer que EXPOSE abre a porta.

Explicação para desenvolver: ENV define um padrão não secreto que pode ser sobrescrito na execução. USER escolhe a identidade do processo; é uma redução de privilégio, não uma garantia de segurança total. EXPOSE registra uma intenção sobre a porta, mas não publica acesso no host. CMD fornece o comando padrão, sujeito a substituição na criação do container. A forma JSON inicia o comando diretamente, sem depender de um shell para interpretar a linha. O servidor é de longa duração, por isso não deve ser iniciado com RUN durante o build.

Transição: “A ordem da cópia também influencia o aproveitamento do cache.”

E4-30 — Lockfile e cache: reproduzir sem reinstalar à toa

Localização: PDF independente, página 31; apresentação principal, posição 137. **Tempo sugerido:** 3 min.

O que o aluno deve levar: Alteração de código não precisa reinstalar dependências quando suas entradas e o cache permanecem válidos.

Fala de abertura sugerida: “Se só o código mudou, pode não haver motivo para reinstalar as mesmas dependências. Por isso copiamos os manifests antes e o código depois.”

Como conduzir: Siga a comparação do desenho. Mostre CACHED se o build já estiver preparado; não edite arquivos dos slides. Diga que cache depende das entradas e de estar disponível.

Explicação para desenvolver: O lockfile registra a resolução das dependências; ele não congela sozinho a base, a plataforma ou todos os componentes externos. Compare as duas ordens do desenho. Quando server.js muda depois de npm ci, a instalação pode ser reutilizada. Se o lockfile muda, a etapa de instalação precisa ser reavaliada. O cache pode não estar disponível numa máquina nova. Demonstre com dois builds e procure CACHED, sem prometer uma duração fixa. Diferencie reutilização de cache de atualização da imagem-base.

O que observar e dizer sobre a saída: CACHED indica reaproveitamento de etapa; ausência dessa palavra numa máquina sem cache não refuta o mecanismo. Para preservar o tempo, use a saída de um build ensaiado ou dois builds consecutivos, sem abrir uma atividade de edição em sala.

Transição: “Além da ordem, precisamos controlar o que enviamos para a construção.”

E4-31 — Contexto de build e .dockerignore

Localização: PDF independente, página 32; apresentação principal, posição 138. **Tempo sugerido:** 2 min.

O que o aluno deve levar: Reprodução e segurança começam no conjunto de arquivos que você permite entrar na construção.

Fala de abertura sugerida: “O ponto no fim do comando de build indica o contexto. O .dockerignore ajuda a impedir que arquivos locais desnecessários façam parte desse envio.”

Como conduzir: Mostre `node_modules`, `.git` e `.env` na lista. Diferencie `.gitignore` de `.dockerignore`; um arquivo fora do Git ainda pode existir na pasta.

Explicação para desenvolver: Use a pasta do laboratório como exemplo de contexto. Um arquivo ignorado pelo Git pode continuar presente no disco e ser enviado ao builder se não for excluído. A cópia seletiva de `server.js` ajuda a evitar inclusão acidental, mas o `.dockerignore` ainda expressa a intenção sobre o contexto. Não copie uma senha e depois tente apagá-la em outra etapa: camadas anteriores podem conservar informação. Valores fictícios do laboratório não devem ser confundidos com gestão de segredos em produção.

Comando ou trecho de apoio: leia a intenção antes de executar. Trechos de configuração podem ser parciais; o arquivo completo do laboratório é a referência.

```
node_modules
.git
.env
.env.*
*.log
README.md
```

Transição: “Vamos construir essa imagem e observar o que ela consegue fazer sem o banco integrado.”

E4-32 — Construir a API e observar sua execução isolada

Localização: PDF independente, página 33; apresentação principal, posição 139. **Tempo sugerido:** 5 min.

O que o aluno deve levar: Uma imagem construída com sucesso ainda precisa ser executada e testada no contexto de suas dependências.

Fala de abertura sugerida: “A API pode estar em execução mesmo sem conseguir acessar o banco. Vamos verificar separadamente a vida do processo HTTP e a prontidão da dependência.”

Como conduzir: Use dois terminais. Deixe `docker run` em primeiro plano no A e faça os `curls` no B. Mostre 200 em `live` e 503 em `ready`. Ao concluir, `Ctrl+C` no A encerra esta instância com `--rm`.

Explicação para desenvolver: Use a porta 18081 livre para não conflitar com a `stack` `Compose`, que usa 8080. A aplicação do exemplo pode iniciar o processo sem conexão prévia com o banco; a consulta de prontidão falha sob demanda. Por isso `live` pode funcionar enquanto `ready` falha. A conexão da `pool` precisa de um `PostgreSQL` alcançável para as operações que o utilizam. A demonstração não prova o sistema inteiro: ela prova que a imagem inicia a API e deixa evidente a dependência ainda ausente.

Comando ou trecho de apoio: leia a intenção antes de executar. Trechos de configuração podem ser parciais; o arquivo completo do laboratório é a referência.

```
docker build -t e4-api:local .
docker run --rm -p 127.0.0.1:18081:3000 e4-api:local

# Em outro terminal:
curl -i http://localhost:18081/health/live
curl -i http://localhost:18081/health/ready
```

O que observar e dizer sobre a saída: Esperado: `live` retorna HTTP 200; `ready` retorna HTTP 503 pela dependência ausente. O terminal A fica ocupado pelo servidor em primeiro plano. Use o B para `curl`. Ao terminar, `Ctrl+C` no A; não deixe a turma achando que o build ficou preso.

Transição: “A imagem que construímos agora pode ser identificada e reutilizada.”

E4-33 — Recursos de imagem: construir, obter e identificar

Localização: PDF independente, página 34; apresentação principal, posição 140. **Tempo sugerido:** 2 min.

O que o aluno deve levar: build produz; pull obtém; tag referencia; push distribui; run cria uma instância.

Fala de abertura sugerida: “Build constrói, pull obtém uma imagem e tag adiciona uma referência. Nenhuma dessas operações, sozinha, prova que o sistema integrado funciona.”

Como conduzir: Escolha uma imagem local para mostrar sua identificação. Explique push sem publicar em conta real. Não faça uma nova construção só para demonstrar outra tag.

Explicação para desenvolver: Diferencie obter uma imagem pronta de construir a própria. Uma tag adicional não reconstrói o conteúdo nem inicia um container. O histórico ajuda a examinar etapas e metadados, mas não é um inventário completo de vulnerabilidades. A operação push publica em um registry autorizado e exige uma referência e permissão adequadas; fica explicada na consulta, sem publicação em contas reais nesta demonstração. A equipe deve saber identificar qual imagem originou a execução testada.

Transição: “Vamos verificar a diferença entre construir e executar.”

E4-34 — Verificação | Imagem e execução

Localização: PDF independente, página 35; apresentação principal, posição 141. **Tempo sugerido:** 2 min.

O que o aluno deve levar: Discuta primeiro. O próximo slide traz as respostas comentadas.

Fala de abertura sugerida: “Respondam considerando o momento em que cada instrução atua. Não basta dizer que “funciona”: expliquem o que aconteceria no build e na partida.”

Como conduzir: Peça respostas rápidas para as três situações. Na última, exija que diferenciem processo respondendo e funcionalidade integrada.

Perguntas e respostas esperadas — consulta do professor; revelar no slide seguinte:

1. **Pergunta:** Trocar RUN npm ci por CMD npm ci mantém o mesmo comportamento?

Resposta esperada: Não. RUN instala durante o build; CMD executaria a instalação na partida, no lugar do comando principal esperado.

2. **Pergunta:** Alterar server.js depois da cópia dos manifests precisa invalidar npm ci em todo build?

Resposta esperada: Não necessariamente. A instalação pode usar cache se suas entradas e a base continuam válidas. O código foi copiado depois.

3. **Pergunta:** Se live retorna 200 e ready retorna 503 nesta execução isolada, o que já foi comprovado?

Resposta esperada: O processo HTTP responde. A consulta de prontidão não passou; sem o banco integrado, ainda não há prova da funcionalidade de visitas.

Transição: “O gabarito mostra o que já foi comprovado e o que ainda falta.”

E4-35 — Respostas comentadas | Imagem e execução

Localização: PDF independente, página 36; apresentação principal, posição 142. **Tempo sugerido:** 3 min.

O que o aluno deve levar: Uma resposta completa identifica o componente envolvido e explica a evidência.

Fala de abertura sugerida: “RUN e CMD não são intercambiáveis. O cache pode preservar a instalação quando suas entradas permanecem válidas. E HTTP vivo ainda não prova acesso funcional ao banco.”

Como conduzir: Corrija as três respostas. Feche dizendo que a próxima etapa resolve a dependência ainda ausente. Anuncie o intervalo de 15 minutos e o horário de retorno.

Correção comentada e retomada se houver erro:

1. **Retome a pergunta:** Trocar RUN npm ci por CMD npm ci mantém o mesmo comportamento?

Explique: Não. RUN instala durante o build; CMD executaria a instalação na partida, no lugar do comando principal esperado.

Se a turma não entendeu: Peça que indiquem quando cada comando executa: construção ou partida. O CMD do servidor seria substituído por npm ci.

2. **Retome a pergunta:** Alterar server.js depois da cópia dos manifests precisa invalidar npm ci em todo build?

Explique: Não necessariamente. A instalação pode usar cache se suas entradas e a base continuam válidas. O código foi copiado depois.

Se a turma não entendeu: Retome a ordem das cópias. A resposta é condicional: cache disponível e entradas relevantes ainda válidas.

3. **Retome a pergunta:** Se live retorna 200 e ready retorna 503 nesta execução isolada, o que já foi comprovado?

Explique: O processo HTTP responde. A consulta de prontidão não passou; sem o banco integrado, ainda não há prova da funcionalidade de visitas.

Se a turma não entendeu: Peça a frase completa: “o processo HTTP responde, mas ainda não provamos a operação que usa o banco”.

Transição: “Na volta, vamos reunir API e banco com Compose.”

Intervalo — minutos 95–110: anuncie o horário de retorno. Confirme que a demonstração isolada foi encerrada no terminal A; prepare o terminal na pasta do laboratório e compose.yaml para a retomada.

5. Integração, rede, dados e saúde

Janela: 110–145 min.

Retomada depois do intervalo: “Antes do intervalo, a API respondia, mas ainda não tinha o banco integrado. Agora vamos declarar os dois serviços e seguir o caminho de uma operação completa.” Tenha o arquivo completo aberto; os slides o dividem apenas para facilitar a explicação.

E4-36 — Compose: declarar o serviço da API

Localização: PDF independente, página 37; apresentação principal, posição 143. **Tempo sugerido:** 2 min.

O que o aluno deve levar: O arquivo permite revisar e compartilhar a configuração que antes ficaria em uma longa linha de docker run.

Fala de abertura sugerida: “Até aqui executamos uma imagem isolada. Agora esta declaração informa como construir e configurar o serviço app, inclusive como o cliente chega até ele.”

Como conduzir: Abra compose.yaml completo, mas destaque somente o trecho da API. Leia a URL de conexão separando host db e porta 5432; as credenciais são fictícias.

Explicação para desenvolver: Retome o conceito de Compose já apresentado na base. Agora leia um serviço de verdade. O arquivo completo do laboratório também contém depends_on e healthcheck; esta lâmina isola os campos de construção, acesso e configuração. build não significa que o banco será instalado dentro da API. DATABASE_URL precisa do hostname db porque a chamada sairá do container da API. Os nomes app e db são escolhas do projeto, usados como identificadores de serviços.

Comando ou trecho de apoio: leia a intenção antes de executar. Trechos de configuração podem ser parciais; o arquivo completo do laboratório é a referência.

```
services:
  app:
    build: .
    ports:
      - "127.0.0.1:8080:3000"
    environment:
      DATABASE_URL: postgres://aula:aula@db:5432/aula
      HOST: 0.0.0.0
```

Trecho: dependência e saúde aparecem adiante.

Transição: “A outra ponta dessa conexão é um serviço independente.”

E4-37 — Compose: banco de dados em outro serviço

Localização: PDF independente, página 38; apresentação principal, posição 144. **Tempo sugerido:** 2 min.

O que o aluno deve levar: API e banco podem ser atualizados ou recriados separadamente; os dados não devem depender da camada gravável do banco.

Fala de abertura sugerida: “O banco usa uma imagem pronta e guarda seu estado num volume. A API é construída a partir do nosso código; são estratégias diferentes dentro da mesma aplicação.”

Como conduzir: Aponte image no banco e build na API. Mostre dados no serviço e sua declaração no final do arquivo. Não acrescente uma porta pública ao banco.

Explicação para desenvolver: O banco usa uma imagem pronta, enquanto a API tem uma imagem construída no projeto. O volume nomeado dados é montado no diretório esperado por PostgreSQL 16. Versões maiores podem mudar requisitos e demandar migração; não generalize o caminho sem verificar a imagem. A conta aula da inicialização simplifica a aula e não representa privilégios mínimos de produção. O arquivo completo monta init.sql e declara healthcheck; os próximos slides mostram essas responsabilidades.

Comando ou trecho de apoio: leia a intenção antes de executar. Trechos de configuração podem ser parciais; o arquivo completo do laboratório é a referência.

```
db:
  image: postgres:16-alpine
  environment:
    POSTGRES_USER: aula
    POSTGRES_PASSWORD: aula
    POSTGRES_DB: aula
  volumes:
    - dados:/var/lib/postgresql/data
volumes:
  dados:
```

Transição: “Além de iniciar o banco, precisamos preparar o schema inicial.”

E4-38 — Inicializar o contador não é migrar o banco

Localização: PDF independente, página 39; apresentação principal, posição 145. **Tempo sugerido:** 1 min.

O que o aluno deve levar: Recriar um container e recriar o estado do banco são operações diferentes.

Fala de abertura sugerida: “Este SQL cria o contador na primeira inicialização de um diretório de dados vazio. Ele não é executado como uma migração automática toda vez que mudamos o arquivo.”

Como conduzir: Mostre init.sql e sua montagem somente leitura no arquivo completo. Diga o que muda quando o volume já contém dados.

Explicação para desenvolver: O arquivo completo usa um bind mount somente leitura para levar init.sql a docker-entrypoint-initdb.d. A imagem oficial executa scripts dessa pasta ao inicializar um diretório de dados vazio. Se o volume já contém o banco, não há uma nova inicialização equivalente. O mesmo raciocínio vale para variáveis de senha: alterar o YAML não troca por si só a senha persistida. Não apague dados apenas para ocultar a falta de uma migração. No projeto, documente como o schema é criado e evolui.

Comando ou trecho de apoio: leia a intenção antes de executar. Trechos de configuração podem ser parciais; o arquivo completo do laboratório é a referência.

```
CREATE TABLE IF NOT EXISTS contador (
  id integer PRIMARY KEY,
  total integer NOT NULL
);
INSERT INTO contador (id, total)
VALUES (1, 0) ON CONFLICT (id) DO NOTHING;
```

Transição: “Com os dois serviços definidos, vamos seguir o percurso de uma requisição.”

E4-39 — A requisição percorre duas conexões

Localização: PDF independente, página 40; apresentação principal, posição 146. **Tempo sugerido:** 3 min.

O que o aluno deve levar: localhost pertence ao ambiente de quem faz a chamada; dentro da API ele não significa “o container do banco”.

Fala de abertura sugerida: “Existem duas conexões: cliente para API e API para banco. Elas têm origens, destinos e portas diferentes. O endereço deve ser interpretado a partir de quem inicia cada conexão.”

Como conduzir: Trace `host:8080 → app:3000 → db:5432` com o dedo ou no quadro. Peça que acompanhem a mudança de origem, sem alterar nenhuma porta.

Explicação para desenvolver: Trace a chamada com o dedo. O cliente no host chega a `localhost:8080`. O encaminhamento leva ao processo da API na porta 3000. A consulta SQL é uma nova conexão, feita pela API em direção a `db:5432`. `db` é resolvido pelo nome do serviço na rede compartilhada. Não é necessário publicar a porta do banco para que a API a alcance. Em configurações com redes distintas, a descoberta depende de ambos os serviços compartilharem uma rede apropriada.

Transição: “Isso ajuda a separar três configurações que costumam ser confundidas.”

E4-40 — Porta documentada, publicada e interface de escuta

Localização: PDF independente, página 41; apresentação principal, posição 147. **Tempo sugerido:** 2 min.

O que o aluno deve levar: Trocar 8080 por 9090 muda o endereço do cliente; a comunicação da API com `db:5432` continua igual.

Fala de abertura sugerida: “EXPOSE documenta a porta. Ports publica uma entrada no host. A interface de escuta define onde o processo recebe dentro do container. Os três têm funções diferentes.”

Como conduzir: Use três rótulos no quadro: documento, encaminhamento e escuta. Relacione 127.0.0.1 no host e 0.0.0.0 no processo às respectivas camadas.

Explicação para desenvolver: Use o primeiro servidor da aula para tornar a distinção concreta. O host estava em 18080 e o processo na porta 3000; não era necessário que os números fossem iguais. No laboratório integrado usamos 8080 para o cliente. A publicação 127.0.0.1:8080:3000 restringe o acesso ao host local, enquanto `HOST=0.0.0.0` permite ao processo receber pela interface do container. Essas duas configurações atuam em camadas diferentes e não se contradizem.

Transição: “O endereço do banco e outras escolhas também podem variar na execução.”

E4-41 — Configuração de execução fora da imagem

Localização: PDF independente, página 42; apresentação principal, posição 148. **Tempo sugerido:** 2 min.

O que o aluno deve levar: Após mudar `HOST` no YAML: `docker compose up -d --no-deps --force-recreate --wait app`.

Fala de abertura sugerida: “Podemos reutilizar a imagem e fornecer configurações ao iniciar a instância. Mas uma variável usada pelo Compose para montar o YAML não entra automaticamente no processo da aplicação.”

Como conduzir: Explique um exemplo não secreto de porta do host. Diferencie interpolação do Compose de environment da aplicação. Mantenha as credenciais reais fora da projeção.

Explicação para desenvolver: Explique com um valor não secreto: no .env, PORTA_HOST=9090; no YAML, ports pode referenciar \${PORTA_HOST}:3000. Isso configura a publicação, mas não implica que process.env.PORTA_HOST exista na API. Para entregar uma variável ao processo, declare environment ou um mecanismo apropriado como env_file. O laboratório usa credenciais fictícias para ficar autocontido; os projetos devem documentar variáveis sem versionar segredos reais. Valores embutidos no build de certos frontends estáticos exigem reconstrução, mesmo quando parecem “configuração”.

Transição: “Também podemos limitar o acesso de que esse processo precisa.”

E4.42 — Recursos para reduzir acesso desnecessário

Localização: PDF independente, página 43; apresentação principal, posição 149. **Tempo sugerido:** 1 min.

O que o aluno deve levar: Permitir somente o necessário exige conhecer os arquivos, conexões e permissões da aplicação.

Fala de abertura sugerida: “O processo não precisa receber todos os privilégios disponíveis. Já usamos usuário node e uma montagem somente leitura para o SQL inicial; outras restrições dependem do comportamento da aplicação.”

Como conduzir: Aponte medidas presentes e trate read_only e secrets como possibilidades a avaliar. Não aplique novas restrições durante a demonstração principal.

Explicação para desenvolver: Use duas medidas já presentes como ponto de partida: usuário node na API e montagem somente leitura de init.sql. read_only não transforma automaticamente todos os volumes em somente leitura, e aplicações que gravam caches ou temporários precisam de destinos adequados. Secrets no Compose local não equivale a um cofre remoto nem implica criptografia automática do arquivo de origem; controle quem acessa o host e o arquivo. Essas escolhas diminuem acessos desnecessários, mas exigem teste funcional. Não prometa segurança absoluta com uma opção.

Transição: “Agora vamos separar o que pertence à instância do que precisa continuar existindo.”

E4.43 — Arquivos da instância, volume e bind mount

Localização: PDF independente, página 44; apresentação principal, posição 150. **Tempo sugerido:** 2 min.

O que o aluno deve levar: Estado de negócio deve ter persistência planejada; uma imagem nova não deve apagar o banco.

Fala de abertura sugerida: “A camada gravável acompanha a instância. O volume pode ser reutilizado quando ela é substituída. O bind mount aponta para um caminho do host.”

Como conduzir: Associe dados ao volume e init.sql ao bind mount. Anuncie que a prova virá pela comparação do contador antes e depois da recriação.

Explicação para desenvolver: A camada gravável pertence à instância e desaparece quando ela é removida. Um volume nomeado é um recurso gerenciado pelo Docker, reutilizável por uma instância posterior. Um bind mount aponta para um caminho do host e depende de seu conteúdo e permissões. No laboratório, dados é volume nomeado e init.sql é um bind mount somente leitura. Stop/start conserva a mesma instância; down/up cria outra. Nenhuma dessas escolhas substitui backup e um teste de restauração.

Transição: “Mesmo com dados e rede definidos, precisamos saber o que significa “estar funcionando”.”

E4-44 — Estado do processo, prontidão e funcionalidade

Localização: PDF independente, página 45; apresentação principal, posição 151. **Tempo sugerido:** 2 min.

O que o aluno deve levar: Um estado verde vale somente o que a verificação executada cobre.

Fala de abertura sugerida: “Up, healthy e operação funcional são evidências de contratos diferentes. O processo pode estar vivo, o banco pode aceitar uma consulta básica e ainda faltar uma tabela necessária.”

Como conduzir: Relacione ps, /health/ready e /visitas. Diga explicitamente que SELECT 1 não testa a tabela contador.

Explicação para desenvolver: Um processo em execução pode estar incapaz de atender porque o banco caiu. Um healthcheck pode ser limitado: nosso SELECT 1 confirma uma consulta básica, mas não prova que a tabela contador existe. A operação GET/POST /visitas testa outro contrato. Docker registra healthy ou unhealthy conforme o comando configurado; não reinicia automaticamente apenas porque o check falhou. Políticas de restart tratam a saída do processo conforme suas regras. O diagnóstico precisa diferenciar esses fatos.

Transição: “O healthcheck torna uma observação específica repetível.”

E4-45 — Healthcheck: comando e parâmetros de observação

Localização: PDF independente, página 46; apresentação principal, posição 152. **Tempo sugerido:** 1 min.

O que o aluno deve levar: O healthcheck da API consulta /health/ready; o teste funcional ainda precisa executar /visitas.

Fala de abertura sugerida: “O comando de saúde é executado em intervalos. Timeout limita a tentativa e retries participa da marcação de falha. Start period trata o período inicial.”

Como conduzir: Leia um parâmetro por vez e mostre a unidade. Não conte segundos para prometer o instante exato de unhealthy. O check observa, não corrige.

Explicação para desenvolver: Leia as unidades e o comando sem supor familiaridade prévia. pg_isready indica o estado de aceitação de conexões, mas não comprova a credencial e todas as operações da aplicação. A API tem um check próprio no arquivo completo, usando Node para chamar /health/ready. start_period tolera falhas iniciais; um sucesso nesse período já sinaliza a inicialização bem-sucedida. Evite prometer um instante exato para unhealthy, pois duração e agendamento das verificações influenciam. O comando de saúde observa; não corrige a causa.

Comando ou trecho de apoio: leia a intenção antes de executar. Trechos de configuração podem ser parciais; o arquivo completo do laboratório é a referência.

```
healthcheck:
  test: ["CMD-SHELL",
    "pg_isready -U aula -d aula"]
  interval: 5s
  timeout: 3s
  retries: 10
  start_period: 10s
```

Transição: “Esse resultado pode orientar a partida da aplicação dependente.”

E4-46 — depends_on: coordenar a partida dos serviços

Localização: PDF independente, página 47; apresentação principal, posição 153. **Tempo sugerido:** 1 min.

O que o aluno deve levar: Inicialização coordenada ajuda a começar; tratamento de falhas ajuda a continuar operando.

Fala de abertura sugerida: “A condição service_healthy faz a partida depender da saúde declarada do banco. Ela não garante que a dependência nunca vai falhar depois.”

Como conduzir: Aponte somente condition. Antecipe que pararemos db mais adiante para observar esse limite, sem mexer no arquivo agora.

Explicação para desenvolver: Mostre a linha condition: service_healthy. Sem uma condição de saúde apropriada, ordenar a partida não garante que o banco esteja pronto para receber consultas. A espera inicial também não é um contrato de disponibilidade eterna. Na API fornecida, falhas de consulta geram 503 e novas requisições podem funcionar quando a dependência volta. Repetir automaticamente a mesma gravação é outro problema: é preciso avaliar se o efeito já ocorreu e se a operação é idempotente.

Comando ou trecho de apoio: leia a intenção antes de executar. Trechos de configuração podem ser parciais; o arquivo completo do laboratório é a referência.

```
services:
  app:
    # Demais campos definidos anteriormente
    depends_on:
      db:
        condition: service_healthy
```

Transição: “Além de prontidão, a configuração pode controlar consumo de recursos.”

E4-47 — Limites de CPU e memória: controlar o consumo

Localização: PDF independente, página 48; apresentação principal, posição 154. **Tempo sugerido:** 1 min.

O que o aluno deve levar: Medir → definir limite → testar carga e comportamento; copiar números não é dimensionar.

Fala de abertura sugerida: “Limites de CPU e memória são escolhas de execução. Estes valores são exemplos de sintaxe, não recomendações universais para todo projeto.”

Como conduzir: Mostre cpus e mem_limit, relacionando com stats. Não provoque falta de memória; destaque que a medição do projeto orienta o valor adequado.

Explicação para desenvolver: Sem limites configurados, não presuma uma divisão automática justa por container. Os valores ilustram a sintaxe; devem ser medidos para a aplicação real. A CPU pode sofrer throttling ao atingir a quota. Para memória, swap depende de configuração adicional e do host, e não deve ser ignorada no dimensionamento. O laboratório básico não

exige provocar falta de memória. Compare a configuração declarada com uma amostra de consumo e explique que uma única amostra não revela todos os picos.

Comando ou trecho de apoio: leia a intenção antes de executar. Trechos de configuração podem ser parciais; o arquivo completo do laboratório é a referência.

```
services:
  app:
    # Junto dos demais campos do serviço
    cpus: 0.50
    mem_limit: 256m

# Observar depois de aplicar a configuração:
# docker compose stats --no-stream
```

Transição: “E se o processo realmente encerrar? A política de reinício trata esse evento.”

E4-48 — Políticas de reinício: reagir à saída do processo

Localização: PDF independente, página 49; apresentação principal, posição 155. **Tempo sugerido:** 1 min.

O que o aluno deve levar: Restart não é healthcheck, não recupera dados perdidos e não oferece alta disponibilidade entre hosts.

Fala de abertura sugerida: “Restart observa a saída do processo conforme a política. Um processo vivo marcado como unhealthy não é automaticamente reiniciado apenas por isso.”

Como conduzir: Compare serviço contínuo e tarefa que deve terminar. Use o caso de senha errada para mostrar que reiniciar não corrige a configuração.

Explicação para desenvolver: A política observa o encerramento do processo. Um container unhealthy com o processo vivo não é automaticamente reiniciado por essa configuração. on-failure não equivale a inicializar automaticamente após reinício do daemon. Distinguir uma aplicação de longa duração de um job que deve terminar evita reinícios indesejados. As políticas não corrigem senha, schema ou hostname incorretos, e reiniciar repetidamente pode esconder o sintoma. O exemplo é explicativo: a API de laboratório não precisa receber uma política nova para cumprir seu objetivo.

Transição: “Vamos executar o arquivo completo e reunir as evidências.”

E4-49 — Subir a aplicação integrada e verificar a resposta

Localização: PDF independente, página 50; apresentação principal, posição 156. **Tempo sugerido:** 4 min.

O que o aluno deve levar: A API só está integrada quando a operação real consegue atravessar a conexão com o banco.

Fala de abertura sugerida: “Primeiro validamos a declaração, depois subimos a aplicação e, por fim, testamos seu comportamento. Cada passo responde a uma pergunta diferente.”

Como conduzir: Execute `config --quiet, up` e `ps`. Faça `live`, `ready`, `POST` e `GET`. Se `--wait` falhar, interrompa a sequência e investigue antes de declarar sucesso.

Explicação para desenvolver: Reúna os trechos mostrando o arquivo completo, sem pedir aos alunos que montem manualmente um YAML a partir das lâminas. Execute `config --quiet` e depois `up`. Se `--wait` falhar, consulte `ps -a` e logs antes de seguir; esse resultado não é uma autorização para apagar dados. O primeiro build pode exigir downloads. O README inclui alternativas para

investigar porta ocupada. Depois que a stack estiver saudável, execute as rotas e a experiência de persistência apresentada na sequência.

Comando ou trecho de apoio: leia a intenção antes de executar. Trechos de configuração podem ser parciais; o arquivo completo do laboratório é a referência.

```
docker compose config --quiet
docker compose up -d --build --wait
docker compose ps
curl -fsS http://localhost:8080/health/live
curl -fsS http://localhost:8080/health/ready
curl -fsS -X POST http://localhost:8080/visitas
curl -fsS http://localhost:8080/visitas
```

O que observar e dizer sobre a saída: config --quiet: silêncio e código zero indicam validação bem-sucedida. up --wait: aguarda condições de saúde. live: alive; ready: ready; POST e GET: total lido após a gravação. O número real pode ser maior que 1. Se --wait falhar, pare a sequência de sucesso e leia ps -a/logs.

Transição: “Vamos interpretar o que essas saídas realmente demonstraram.”

E4-50 — Interpretar as saídas da demonstração

Localização: PDF independente, página 51; apresentação principal, posição 157. **Tempo sugerido:** 2 min.

O que o aluno deve levar: Comandos e resultados devem ser relacionados a uma afirmação verificável sobre o projeto.

Fala de abertura sugerida: “O importante não é obter exatamente o número 1. É mostrar que a gravação produziu um valor que a leitura recupera. Um volume usado antes pode começar em outro total.”

Como conduzir: Anote o total real N no quadro. Relacione cada saída ao comando que a produziu. Evite tratar tudo como um único “teste verde”.

Explicação para desenvolver: Use os valores da execução real, e não apenas a captura da aula. Leia o total antes da gravação e compare com o resultado do POST. Para demonstrar a persistência, use a sequência down/up sem -v e faça a nova leitura. A evidência deve descrever qual comportamento foi comprovado, não apenas que algo ficou verde. SELECT 1 bem-sucedido não prova schema, permissões de todas as tabelas nem toda a lógica de negócio. Essa distinção prepara o diagnóstico seguinte.

O que observar e dizer sobre a saída: Escreva “N = valor observado” no quadro. Um aluno pode narrar qual saída prova vida, consulta básica e operação de negócio. Resposta esperada: são contratos diferentes; POST/GET acrescenta evidência funcional de escrita e leitura.

Transição: “Agora vamos substituir as instâncias e conferir se N continua disponível.”

E4-51 — Comprovar persistência com uma experiência

Localização: PDF independente, página 52; apresentação principal, posição 158. **Tempo sugerido:** 4 min.

O que o aluno deve levar: Persistência além da instância não é recuperação de desastre; volume e backup resolvem problemas diferentes.

Fala de abertura sugerida: “Vamos remover os containers desta stack preservando o volume, subir de novo e ler o mesmo dado. O critério é comparar a última leitura antes com a primeira depois.”

Como conduzir: Execute a sequência sem -v. Não faça outro POST entre as leituras comparadas. Mantenha a mesma pasta e nome de projeto para reutilizar o mesmo volume.

Explicação para desenvolver: Faça a demonstração com dados fictícios. Os números dependem do estado anterior do volume, então não exija total igual a 1. A propriedade testada é a igualdade da última leitura antes da parada com a leitura após a recriação. Um nome de projeto diferente pode criar outro volume; um contador zerado nesse caso não prova que o anterior foi perdido. `down -v` remove volumes declarados e não é o encerramento padrão do laboratório. Perda do disco do host exige uma estratégia de backup além deste teste.

Comando ou trecho de apoio: leia a intenção antes de executar. Trechos de configuração podem ser parciais; o arquivo completo do laboratório é a referência.

```
curl -fsS -X POST http://localhost:8080/visitas
curl -fsS http://localhost:8080/visitas
docker compose down
docker compose up -d --wait
curl -fsS http://localhost:8080/visitas
```

O que observar e dizer sobre a saída: A última leitura antes de `down` deve ser igual à leitura depois de `up`, sem um novo POST entre elas. Se mudou, confira a intervenção e se o mesmo projeto/volume foi reutilizado antes de concluir perda de dados.

Transição: “Vamos verificar se conseguimos distinguir rede, persistência e saúde.”

E4-52 — Verificação | Rede, dados e saúde

Localização: PDF independente, página 53; apresentação principal, posição 159. **Tempo sugerido:** 2 min.

O que o aluno deve levar: Discuta primeiro. O próximo slide traz as respostas comentadas.

Fala de abertura sugerida: “Pensem em três mudanças independentes: a entrada do cliente, a vida do volume e o alcance do healthcheck. Usem o desenho e o experimento para justificar.”

Como conduzir: Dê uma resposta por situação e peça a justificativa. Observe especialmente quem muda a porta do banco junto com a porta do host.

Perguntas e respostas esperadas — consulta do professor; revelar no slide seguinte:

1. **Pergunta:** A porta do host muda para 9090. A API deve passar a usar db:9090?

Resposta esperada: Não. 9090 é a entrada do cliente na API. A conexão interna com o banco continua db:5432.

2. **Pergunta:** Depois de `down/up` sem -v, o total permanece. Isso prova que há backup?

Resposta esperada: Não. Prova persistência no volume reutilizado. Backup requer uma cópia recuperável e restauração testada.

3. **Pergunta:** O check `SELECT 1` passa, mas a tabela contador não existe. `/visitas` está garantido?

Resposta esperada: Não. A consulta básica pode passar enquanto a operação falha por schema ausente. Teste `GET/POST /visitas`.

Transição: “O gabarito retoma exatamente qual propriedade cada teste demonstra.”

E4.53 — Respostas comentadas | Rede, dados e saúde

Localização: PDF independente, página 54; apresentação principal, posição 160. **Tempo sugerido:** 2 min.

O que o aluno deve levar: Uma resposta completa identifica o componente envolvido e explica a evidência.

Fala de abertura sugerida: “Mudar a entrada do cliente não muda a porta interna do banco. Preservar um volume não comprova backup. Uma consulta básica não garante toda a operação.”

Como conduzir: Se necessário, volte ao fluxo de E4.39 e ao contador de E4.51. Corrija sem abrir uma nova configuração ou outro laboratório.

Correção comentada e retomada se houver erro:

1. **Retome a pergunta:** A porta do host muda para 9090. A API deve passar a usar db:9090?

Explique: Não. 9090 é a entrada do cliente na API. A conexão interna com o banco continua db:5432.

Se a turma não entendeu: Aponte duas setas no fluxo. A porta pública da API pertence à primeira; db:5432 pertence à segunda.

2. **Retome a pergunta:** Depois de down/up sem -v, o total permanece. Isso prova que há backup?

Explique: Não. Prova persistência no volume reutilizado. Backup requer uma cópia recuperável e restauração testada.

Se a turma não entendeu: Pergunte se os dados sobreviveriam à perda do disco sem outra cópia. Persistência local e backup têm objetivos distintos.

3. **Retome a pergunta:** O check SELECT 1 passa, mas a tabela contador não existe. /visitas está garantido?

Explique: Não. A consulta básica pode passar enquanto a operação falha por schema ausente. Teste GET/POST /visitas.

Se a turma não entendeu: Compare SELECT 1 com a consulta à tabela contador. Só a operação funcional alcança esse requisito.

Transição: “Com essa base, conseguimos investigar falhas com método.”

6. Diagnóstico guiado e uso crítico de IA

Janela: 145–165 min.

E4-54 — Diagnóstico: observar antes de alterar

Localização: PDF independente, página 55; apresentação principal, posição 161. **Tempo sugerido:** 2 min.

O que o aluno deve levar: Sintoma → hipótese → teste → correção mínima → nova validação → registro.

Fala de abertura sugerida: “Antes de mudar uma configuração, identifique em que etapa ocorreu a falha e registre o sintoma. Depois formule uma hipótese e escolha uma verificação que possa contrariá-la.”

Como conduzir: Escreva no quadro sintoma → hipótese → teste → mudança → repetição. Explique por que mudar cinco coisas de uma vez impede aprender a causa.

Explicação para desenvolver: Comece identificando se o problema ocorreu no build ou na execução. No build, confira o contexto, os arquivos e a etapa que falhou. Na execução, inclua containers parados na inspeção, leia logs e só então relacione configuração, rede e dependências. Uma mensagem de conexão recusada sugere que não houve serviço aceitando naquele destino; não prova sozinha por que isso ocorreu. Mudar várias coisas simultaneamente impede saber qual hipótese estava correta.

Transição: “Os comandos de diagnóstico entram para responder a perguntas específicas.”

E4-55 — Comandos com uma pergunta técnica definida

Localização: PDF independente, página 56; apresentação principal, posição 162. **Tempo sugerido:** 2 min.

O que o aluno deve levar: Não memorize uma lista de comandos sem entender qual hipótese cada um testa.

Fala de abertura sugerida: “Se a instância parou, exec não é o primeiro passo. Precisamos ver seu estado e suas últimas saídas. Se está ativa, podemos testar um ponto específico de dentro dela.”

Como conduzir: Mostre a ordem ps -a, logs e depois exec. Inspecione apenas HOST se precisar de exemplo; não projete todas as variáveis de ambiente.

Explicação para desenvolver: Explique por que exec não é o primeiro recurso se a instância encerrou: ele precisa de um container ativo. Primeiro veja ps -a e logs. O comando node que imprime HOST é um exemplo seguro porque inspeciona apenas um valor não secreto. Evite imprimir toda a configuração ou environment em um ambiente real. Em imagens mínimas, curl e shell podem não existir; no laboratório há Node, então é possível usar suas bibliotecas para verificações específicas.

Transição: “Vamos aplicar esse raciocínio a um erro de endereço.”

E4-56 — Caso resolvido: banco saudável, API retorna 503

Localização: PDF independente, página 57; apresentação principal, posição 163. **Tempo sugerido:** 3 min.

O que o aluno deve levar: A evidência antes/depois deve confirmar a causa; apagar o volume não testa esta hipótese.

Fala de abertura sugerida: “O banco pode estar saudável e a API procurar o destino errado. Dentro da API, localhost aponta para ela mesma; não passa a significar outro container só porque ambos estão no Compose.”

Como conduzir: Use o caso já resolvido no slide. Trace a conexão errada e a correta; não provoque esta edição ao vivo se isso consumir o tempo da falha principal.

Explicação para desenvolver: Use apenas a cópia de laboratório para provocar a falha. Altere uma coisa: o hostname da URL. Recrie app sem --wait, pois estamos esperando uma falha de prontidão, e recolha logs e resposta HTTP. Confirme que db continua disponível. A correção é restaurar db na URL e recriar a instância, mantendo volume e banco. O README tem os comandos completos de provocação, inspeção e recuperação. Publicar 5432 no host não corrige o significado de localhost dentro da API.

Transição: “Outro caso depende do ponto de onde fazemos o teste.”

E4-57 — Caso resolvido: check verde, acesso externo falha

Localização: PDF independente, página 58; apresentação principal, posição 164. **Tempo sugerido:** 3 min.

O que o aluno deve levar: Um teste interno bem-sucedido não garante o caminho percorrido pelo cliente externo.

Fala de abertura sugerida: “Uma chamada interna pode funcionar enquanto o acesso pelo host falha. Precisamos verificar em qual interface o processo está escutando e de onde partiu a observação.”

Como conduzir: Compare loopback interno e acesso publicado. Mostre que um healthcheck interno verde pode não cobrir o caminho externo. Use o caso resolvido como contraste.

Explicação para desenvolver: Este caso mostra por que definir de onde o teste parte é essencial. O healthcheck da aplicação usa o loopback interno, que continua acessível com o bind errado para tráfego externo. Antes de corrigir, demonstre os dois pontos de observação. Em seguida mude apenas HOST, recrie e repita as chamadas. O arquivo da imagem EXPOSE não altera a interface real de escuta do processo. O mapeamento no Compose precisa continuar apontando para a porta correta.

Transição: “Agora vamos provocar uma queda de dependência sem editar o YAML.”

E4-58 — Caso resolvido: dependência cai durante o uso

Localização: PDF independente, página 59; apresentação principal, posição 165. **Tempo sugerido:** 7 min.

O que o aluno deve levar: Recuperar a dependência e testar novas requisições pode bastar; a decisão deve seguir a evidência.

Fala de abertura sugerida: “Vou parar somente o banco e repetir os mesmos testes. A hipótese é que o processo HTTP continue vivo, mas a operação dependente do banco deixe de funcionar.”

Como conduzir: Execute a sequência de falha deste roteiro. Registre 200 em live e 503 em ready/visitas. Retome db, aguarde ready voltar a 200 e só então valide a operação e os dados.

Explicação para desenvolver: Esta é a demonstração de falha mais rápida, sem edição do YAML. O código da API trata erros de consulta e da pool para permitir novas requisições após o banco voltar. Isso foi validado no laboratório anterior e deve ser ensaiado no ambiente da aula. Não generalize a recuperação para toda aplicação: algumas precisam de tratamento adicional. Discuta por que reiniciar tudo pode ser desnecessário e também esconder evidências úteis.

O que observar e dizer sobre a saída: Antes de parar db, registre GET /visitas. Com db parado: live 200; ready e visitas 503. Após start, aguarde o banco e a API recuperarem prontidão; respostas transitórias de erro podem ocorrer. Só declare recuperação após ready 200 e GET funcional com o total preservado.

Sequência de terminal para esta demonstração (pasta do laboratório E4):

```
curl -fsS http://localhost:8080/visitas
docker compose stop db
curl -sS -i http://localhost:8080/health/live
curl -sS -i http://localhost:8080/health/ready
curl -sS -i http://localhost:8080/visitas
docker compose logs --tail=30 app
docker compose start db
docker compose exec db pg_isready -U aula -d aula
curl -sS -i http://localhost:8080/health/ready
# Aguardar e repetir ready até HTTP 200 antes de prosseguir.
curl -fsS http://localhost:8080/visitas
```

Ritmo dos sete minutos: 1 min para referência e hipótese; 2 min para falha e sinais; 2 min para iniciar recuperação e aguardar; 2 min para validar e sintetizar. Se a recuperação demorar, explique o estado observado e use o plano de contingência; não diga que recuperou sem verificar. Nunca use remoção de volume para esta falha.

Transição: “A IA pode ajudar a organizar hipóteses, desde que a equipe saiba testá-las.”

E4-59 — IA no diagnóstico: hipótese com teste verificável

Localização: PDF independente, página 60; apresentação principal, posição 166. **Tempo sugerido:** 3 min.

O que o aluno deve levar: Toda saída de IA é hipótese até ser validada por teste, execução ou revisão humana.

Fala de abertura sugerida: “Uma explicação plausível da IA não é a evidência da causa. Peçam hipóteses com verificação e resultado esperado; depois comparem com o sistema observado.”

Como conduzir: Leia o prompt do laboratório. Use como exemplo hipotético a sugestão de apagar dados para resolver porta errada e explique por que deve ser rejeitada.

Explicação para desenvolver: A IA entra após os alunos entenderem o sistema que estão diagnosticando. Ela pode organizar hipóteses e explicar mensagens, mas a hipótese só é útil se puder ser confrontada com uma evidência. Mostre uma recomendação sem relação causal, como apagar um volume para corrigir uma porta, e explique por que ela é rejeitada. Não exponha dumps de environment ou inspect contendo segredos. Um registro crítico inclui sugestões rejeitadas, não apenas as que funcionaram.

Transição: “Vamos aplicar o mesmo raciocínio ao projeto de cada equipe.”

7. Trabalho das equipes no projeto

Janela: 165–235 min.

Como usar estes oito slides: apresente E4-60–62 em até seis minutos, enquanto os grupos já localizam os arquivos. Deixe E4-63 como tela de trabalho. Use E4-64–65 na circulação, sem interromper todas as equipes a cada orientação. Faça E4-66–67 na revisão final, dentro do mesmo bloco.

Relógio da aula	Foco das equipes	Sua intervenção
165–185	Empacotar (0–20 min do bloco).	Nos primeiros 6 min, orientar E4-60–62; depois conferir comando, runtime e contexto.
185–210	Integrar (20–45 min do bloco).	Aplicar E4-64 como checklist: rede, configuração, operação e estado.
210–225	Diagnosticar e registrar.	Usar o modelo E4-65; conferir uma hipótese, uma mudança e um teste repetido.
225–233	Revisar explicações e evidências.	E4-66–67: respostas dos grupos e gabarito, até 4 min cada.
233–235	Consolidar o registro.	Salvar instruções, saídas, links e pendências reais.

Três perguntas na circulação, com o que esperar: “Qual processo precisa iniciar?” → comando e runtime concretos. “Quem chama quem?” → origem, hostname e porta do destino. “Qual saída comprova a operação?” → resultado funcional esperado, não só um container Up.

E4-60 — Como este encontro entra no projeto integrador

Localização: PDF independente, página 61; apresentação principal, posição 167. **Tempo sugerido:** dentro do trabalho das equipes; veja o plano do bloco.

O que o aluno deve levar: E4 alimenta os 20% de Containers/Compose já previstos na rubrica; os demais pesos permanecem os oficiais.

Fala de abertura sugerida: “Vocês continuam o projeto que já escolheram. Hoje acrescentam uma forma documentada de executar a aplicação e suas dependências, conectada ao trabalho de CI já existente.”

Como conduzir: Retome o repositório de cada grupo e a entrega de Containers/Compose, mantendo o peso oficial de 20%. Não abra uma segunda atividade com outra aplicação.

Explicação para desenvolver: Agora faça a ligação com os encontros anteriores. A equipe já escolheu uma aplicação, organizou colaboração e criou CI. O incremento de hoje não substitui nada disso: acrescenta um contrato de execução da aplicação e suas dependências. No próximo encontro, a imagem verificável será candidata a uma entrega controlada, com configuração e rollback. No encontro final, logs e evidências vão sustentar a demonstração. Não há um segundo projeto separado nem obrigação de adotar a API de visitas.

Transição: “O primeiro passo é traduzir as responsabilidades do exemplo para a stack de vocês.”

E4-61 — Traduzir o exemplo para a aplicação da equipe

Localização: PDF independente, página 62; apresentação principal, posição 168. **Tempo sugerido:** dentro do trabalho das equipes; veja o plano do bloco.

O que o aluno deve levar: Antes de escrever o Dockerfile, anote comando, runtime, dependências, configuração e estado do próprio projeto.

Fala de abertura sugerida: “Não copiem comandos de Node para uma aplicação de outra linguagem. Identifiquem qual runtime, comando de partida e dependência correspondem ao projeto de vocês.”

Como conduzir: Peça que cada equipe nomeie seus equivalentes no próprio repositório. Se não usa banco, peça uma justificativa sobre integração e estado; não imponha PostgreSQL.

Explicação para desenvolver: Peça que a equipe identifique equivalentes no próprio repositório. Uma aplicação Java pode executar um jar, Python um módulo ou servidor WSGI/ASGI, e Node um arquivo ou script definido. Não copie npm ci para outra stack. Dependência pode ser PostgreSQL, outro banco ou um serviço auxiliar pertinente. Se uma aplicação não precisa de banco, não acrescente um sem justificativa só para imitar o exemplo; explique sua integração e os recursos de estado reais. A evidência deve avaliar a aplicação escolhida, não uma demonstração paralela.

Transição: “Antes de executar, registrem essas decisões de forma objetiva.”

E4-62 — Plano técnico: decisões antes de executar

Localização: PDF independente, página 63; apresentação principal, posição 169. **Tempo sugerido:** dentro do trabalho das equipes; veja o plano do bloco.

O que o aluno deve levar: As decisões do README devem corresponder aos arquivos versionados e à execução que será demonstrada.

Fala de abertura sugerida: “Quero respostas que permitam executar o sistema: qual comando inicia, qual porta recebe, qual nome identifica a dependência, quais variáveis são necessárias e onde os dados ficam.”

Como conduzir: Dê dois minutos para anotar essas decisões. Circule para detectar localhost indevido ou ausência de comando de partida. Use a lista como plano de ação do grupo.

Explicação para desenvolver: Estas perguntas são uma atividade de aplicação, depois da explicação e da demonstração. Oriente a equipe a registrar respostas objetivas, não frases como “usar Docker para padronizar”. Um bom registro diz, por exemplo: API escuta 3000, cliente usa 8080, banco é acessado pelo nome db e dados vivem em um volume. A escolha de mecanismos de configuração e schema deve acompanhar a stack real. A próxima lâmina distribui o trabalho em checkpoints.

Transição: “O trabalho será dividido em empacotar, integrar e validar.”

E4-63 — Laboratório: aplicar ao projeto da equipe

Localização: PDF independente, página 64; apresentação principal, posição 170. **Tempo sugerido:** dentro do trabalho das equipes; veja o plano do bloco.

O que o aluno deve levar: A equipe deve conseguir explicar o que cada serviço faz e reproduzir sua própria execução.

Fala de abertura sugerida: “Organizem funções dentro da equipe: uma pessoa opera, outra acompanha sinais, outra registra e as demais revisam. Alternem os papéis para que todos consigam explicar o caminho.”

Como conduzir: Deixe este slide projetado durante a atividade. Faça três passagens pelas equipes, acompanhando os checkpoints do cronograma. Não transforme cada ajuda em uma palestra para toda a turma.

Explicação para desenvolver: Circule pela turma verificando comportamento em vez de apenas arquivos presentes. Uma pessoa pode operar, outra acompanhar logs, outra registrar a evidência e outras revisar o comando e o critério de sucesso; troquem funções. Mantenha o acompanhamento e a entrega de grupo conforme o regulamento existente. Se o tempo estiver curto, preserve o teste funcional e o registro de uma falha compreendida. O laboratório da API de visitas continua como referência, não como substituto do projeto.

Transição: “Na circulação, vamos conferir o comportamento esperado, não só a presença dos arquivos.”

E4-64 — Critérios de aceite: o que deve ficar demonstrado

Localização: PDF independente, página 65; apresentação principal, posição 171. **Tempo sugerido:** dentro do trabalho das equipes; veja o plano do bloco.

O que o aluno deve levar: Arquivo presente é parte da evidência; resultado reproduzível e explicação da decisão completam a entrega.

Fala de abertura sugerida: “Um Dockerfile e um Compose no repositório são necessários quando pertinentes, mas não bastam. Outra pessoa precisa conseguir seguir as instruções e observar a operação funcionando.”

Como conduzir: Use este slide como checklist na mesa de cada equipe ou em uma pausa breve. Peça execução pelo README sem explicação oral adicional; não apague dados reais para simular clone limpo.

Explicação para desenvolver: Não altere os pesos homologados nem introduza uma nota individual. Este conjunto de evidências contribui para o critério já estabelecido. Clone limpo significa que os arquivos necessários estão disponíveis pelo repositório e pelas instruções, não que dados reais devam ser apagados. Peça que alguém siga o README sem ajuda oral para revelar etapas implícitas. Um print do Docker aberto não prova integração funcional ou diagnóstico.

Transição: “O relato da falha deve permitir que alguém acompanhe o raciocínio.”

E4-65 — Exemplo preenchido: evidência de uma falha corrigida

Localização: PDF independente, página 66; apresentação principal, posição 172. **Tempo sugerido:** dentro do trabalho das equipes; veja o plano do bloco.

O que o aluno deve levar: Substitua os valores do exemplo pela execução real e mantenha toda evidência no repositório.

Fala de abertura sugerida: “Um registro bom diz o que falhou, qual hipótese foi testada, o que mudou e como o mesmo teste passou depois. Este exemplo é um modelo, não a execução de vocês.”

Como conduzir: Mostre o modelo quando os grupos entrarem na fase de diagnóstico. Peça links e saídas da execução real. Diferencie hipótese rejeitada, teste pendente e causa sustentada.

Explicação para desenvolver: Mostre um registro curto mas causal. O importante é que o aluno possa relacionar a hipótese à evidência e à mudança específica. Não apresente os dados desta tabela como execução da equipe; são um modelo didático. Se a equipe usou IA, registre qual hipótese ela sugeriu e como foi aceita ou descartada. A validação final deve testar tanto a prontidão quanto uma operação funcional, porque os contratos são diferentes.

Transição: “Na revisão final, vocês vão explicar a solução usando essas evidências.”

E4-66 — Verificação final | Explicar o próprio projeto

Localização: PDF independente, página 67; apresentação principal, posição 173. **Tempo**

sugerido: dentro do trabalho das equipes; veja o plano do bloco.

O que o aluno deve levar: Discuta primeiro. O próximo slide traz as respostas comentadas.

Fala de abertura sugerida: “Agora expliquem o próprio projeto com os conceitos da aula. Não quero uma definição decorada: mostrem qual é a imagem, o processo e a operação que comprova a integração.”

Como conduzir: Use as três perguntas durante a rodada final de revisão. Peça evidências do grupo; mantenha caráter formativo e avaliação coletiva.

Perguntas e respostas esperadas — consulta do professor; revelar no slide seguinte:

1. **Pergunta:** Ao apresentar o projeto, como distinguir imagem, container e os dois sentidos de runtime?

Resposta esperada: Imagem: artefato. Container: instância. Runtime da linguagem executa o código; runtime de containers prepara e inicia processos isolados.

2. **Pergunta:** Como provar que sua aplicação está integrada à dependência, além de mostrar o container Up?

Resposta esperada: Executar uma operação funcional com resultado esperado; verificar saúde e persistência conforme o contrato do projeto.

3. **Pergunta:** Qual evidência liga uma alteração de configuração à correção de uma falha?

Resposta esperada: Sintoma reproduzido, hipótese testada, mudança específica e repetição do mesmo teste com resultado corrigido.

Transição: “Vamos usar o gabarito como critério de clareza para a explicação.”

E4-67 — Respostas comentadas | Explicar o próprio projeto

Localização: PDF independente, página 68; apresentação principal, posição 174. **Tempo**

sugerido: dentro do trabalho das equipes; veja o plano do bloco.

O que o aluno deve levar: Uma resposta completa identifica o componente envolvido e explica a evidência.

Fala de abertura sugerida: “Uma explicação completa separa artefato e instância, mostra uma operação funcional e liga a correção a um teste repetido. Os nomes e comandos dependem do projeto.”

Como conduzir: Compare as respostas sem exigir que todos usem a mesma stack. Reserve os minutos restantes do bloco para corrigir o README e registrar pendências honestamente.

Correção comentada e retomada se houver erro:

1. **Retome a pergunta:** Ao apresentar o projeto, como distinguir imagem, container e os dois sentidos de runtime?

Explique: Imagem: artefato. Container: instância. Runtime da linguagem executa o código; runtime de containers prepara e inicia processos isolados.

Se a turma não entendeu: Peça exemplos do repositório do grupo: imagem usada, instância ativa e runtime da stack.

2. **Retome a pergunta:** Como provar que sua aplicação está integrada à dependência, além de mostrar o container Up?

Explique: Executar uma operação funcional com resultado esperado; verificar saúde e persistência conforme o contrato do projeto.

Se a turma não entendeu: Peça uma operação com entrada, resultado esperado e saída observada; um print de Up não basta.

3. **Retome a pergunta:** Qual evidência liga uma alteração de configuração à correção de uma falha?

Explique: Sintoma reproduzido, hipótese testada, mudança específica e repetição do mesmo teste com resultado corrigido.

Se a turma não entendeu: Peça que mostrem antes, mudança específica e depois, mantendo o mesmo teste.

Transição: “Vamos fechar ligando o ambiente executável à entrega controlada.”

8. Fechamento da trilha principal

Janela: 235–240 min.

E4-68 — Do ambiente executável à entrega controlada

Localização: PDF independente, página 69; apresentação principal, posição 175. **Tempo sugerido:** 3 min.

O que o aluno deve levar: Containerização tem valor quando melhora a reprodução, a integração e a entrega do projeto.

Fala de abertura sugerida: “Hoje construímos um contrato de execução e aprendemos a demonstrar integração. No próximo encontro, vamos discutir como entregar uma versão conhecida e recuperar uma mudança quando necessário.”

Como conduzir: Peça uma síntese curta de uma equipe: decisão, evidência e dificuldade resolvida. Retome a imagem como artefato que poderá entrar no fluxo de entrega.

Explicação para desenvolver: Feche o encontro retomando a sequência lógica, sem abrir um novo laboratório. A equipe agora tem uma forma de executar a aplicação e pode começar a discutir entregar um artefato conhecido. O build de imagem no CI é uma evolução útil, mas não substitui os testes anteriores. Um pipeline que apenas constrói a imagem não prova que ela consegue iniciar e atender no ambiente de destino. O encontro seguinte discutirá controlar essa passagem com critérios explícitos.

Transição: “As referências e os slides seguintes apoiam aprofundamentos depois da aula.”

E4-69 — Referências e consulta depois da aula

Localização: PDF independente, página 70; apresentação principal, posição 176. **Tempo sugerido:** 2 min.

O que o aluno deve levar: Fim da trilha principal. Os próximos slides são material de consulta, sem nova entrega obrigatória.

Fala de abertura sugerida: “Vocês não precisam adotar todos os recursos do Docker para concluir este incremento. Precisam justificar os que usam e comprovar o comportamento do projeto.”

Como conduzir: Indique o README, os comandos completos e as referências. Confira que os grupos registraram execução, diagnóstico e pendências. Encerre a trilha de quatro horas aqui.

Explicação para desenvolver: As referências oficiais estão com links completos no guia e na apostila. Os diagramas são esquemas didáticos editáveis, não capturas de uma arquitetura obrigatória em todas as instalações. Consulte as versões efetivamente utilizadas quando houver diferença de comando ou implementação. As próximas lâminas são de consulta para aprofundamento, não uma extensão obrigatória da aula de quatro horas.

Transição: “Os próximos nove slides ficam para consulta ou dúvidas específicas.”

9. Slides de consulta: usar conforme a dúvida

Janela: Fora do percurso obrigatório.

Uso opcional: todas as explicações estão aqui para você responder às dúvidas. Não abra estes nove slides como outra sequência obrigatória após o minuto 240. Estime 1–3 minutos por consulta escolhida, usando tempo de dúvidas já disponível ou estudo posterior.

E4-70 — Consulta | Registry, tag e digest

Localização: PDF independente, página 71; apresentação principal, posição 177. **Tempo sugerido:** 1–3 min se houver uma dúvida pertinente.

O que o aluno deve levar: Mesma tag não prova igualdade de conteúdo entre máquinas ou momentos diferentes.

Fala de abertura sugerida: “Registry armazena imagens. Tag é uma referência legível que pode mudar. Digest identifica um conteúdo específico; controlar atualização continua sendo uma decisão do projeto.”

Como conduzir: Use a tag Node já conhecida e relacione com promover o mesmo artefato testado. Não faça push só para ilustrar o conceito.

Explicação para desenvolver: Use node:22-bookworm-slim como exemplo: uma atualização do publicador pode mudar o conteúdo por trás da tag. O lockfile npm não fixa a imagem-base. Fixar digest ajuda a controlar atualização, mas exige política para receber correções. Não faça push para contas reais nesta aula. A distinção é relevante quando o mesmo artefato testado deve ser promovido, assunto do próximo encontro.

Transição: “Se a dúvida for tamanho ou ferramentas de compilação, avance para multi-stage.”

E4-71 — Consulta | Multi-stage e separação de responsabilidades

Localização: PDF independente, página 72; apresentação principal, posição 178. **Tempo sugerido:** 1–3 min se houver uma dúvida pertinente.

O que o aluno deve levar: Multi-stage é uma escolha de empacotamento; use quando houver uma separação útil entre construir e executar.

Fala de abertura sugerida: “Algumas aplicações precisam de ferramentas para construir o artefato que não são necessárias para executá-lo. Multi-stage permite separar essas etapas.”

Como conduzir: Compare TypeScript compilado com o JavaScript simples da aula. Explique quando há benefício, sem exigir mudança no Dockerfile de todos os grupos.

Explicação para desenvolver: A API da aula é JavaScript sem compilação; um Dockerfile simples atende ao objetivo. Num projeto TypeScript, separar a construção permite excluir ferramentas usadas apenas para compilar. Em Java ou Go a forma do artefato será diferente. Isso não autoriza copiar um binário entre bases incompatíveis sem revisar suas bibliotecas. O aluno deve reconhecer quando o padrão é útil, sem ser obrigado a adicioná-lo ao projeto se ele não resolve um problema concreto.

Transição: “Depois de construir, podemos verificar a imagem automaticamente no CI.”

E4-72 — Consulta | Verificar a imagem no CI

Localização: PDF independente, página 73; apresentação principal, posição 179. **Tempo sugerido:** 1–3 min se houver uma dúvida pertinente.

O que o aluno deve levar: CI verde só comprova o que foi testado; executar a imagem amplia a evidência sobre o artefato.

Fala de abertura sugerida: “Um build bem-sucedido é uma evidência, mas ainda podemos precisar iniciar a aplicação e testar uma operação. O CI deve tornar explícito o resultado esperado.”

Como conduzir: Descreva build → partida → asserção → encerramento controlado. Não configure um novo pipeline durante a aula principal.

Explicação para desenvolver: Esta lâmina conecta o laboratório ao pipeline sem impor mais uma entrega no encontro. Um teste de integração em CI precisa de asserções e de tratamento da limpeza em falha; listar compose up seguido de down não garante por si só esse comportamento em qualquer executor. Dê preferência a um projeto Compose exclusivo por job para evitar colisões. Não use prune global ou remoção de volumes reais como rotina de limpeza. A publicação de artefato deve seguir as permissões e políticas do projeto.

Transição: “Outras necessidades aparecem quando escolhemos como armazenar arquivos.”

E4-73 — Consulta | Volume, bind mount e tmpfs

Localização: PDF independente, página 74; apresentação principal, posição 180. **Tempo sugerido:** 1–3 min se houver uma dúvida pertinente.

O que o aluno deve levar: Persistência, compartilhamento com o host e armazenamento temporário são requisitos diferentes.

Fala de abertura sugerida: “Volume, bind mount e tmpfs têm finalidades distintas. Dados que precisam sobreviver à substituição não devem depender apenas de armazenamento temporário.”

Como conduzir: Compare o contador persistente, o SQL do host e um diretório temporário. Relacione à necessidade concreta da equipe que trouxe a dúvida.

Explicação para desenvolver: Um exemplo de sintaxe Compose é tmpfs: /tmp dentro do serviço, útil se a aplicação precisa de temporários numa instância com filesystem somente leitura. Dados de negócio não devem depender dessa montagem. tmpfs usa memória, mas páginas podem ir para swap conforme o host; não prometa que segredos jamais tocarão disco só por estarem ali. O laboratório usa volume para dados e bind mount para inicialização, por isso tmpfs é apresentado como alternativa com propósito distinto.

Transição: “Na rede também existem alternativas com escopos diferentes.”

E4-74 — Consulta | Diferentes modos de rede

Localização: PDF independente, página 75; apresentação principal, posição 181. **Tempo sugerido:** 1–3 min se houver uma dúvida pertinente.

O que o aluno deve levar: Escolha a topologia necessária antes do driver; uma rede compartilhada não publica sozinha portas no host.

Fala de abertura sugerida: “Os drivers de rede não são níveis de qualidade. Cada um atende a um contexto; a bridge do exemplo já permite a comunicação necessária em um host.”

Como conduzir: Volte primeiro à pergunta “quem precisa alcançar quem?”. Não proponha overlay ou host para corrigir um nome de serviço errado.

Explicação para desenvolver: Não apresente drivers como uma progressão de qualidade. Bridge atende ao exemplo de dois serviços em um host. Host reduz o isolamento de rede e não exige publicação de portas da mesma forma. Overlay não é o resultado automático de executar Compose em dois notebooks. Macvlan e ipvlan atendem necessidades específicas de infraestrutura e podem ter restrições no ambiente local. No projeto, a primeira decisão é quais serviços precisam se alcançar; trocar de driver sem uma necessidade não corrige um hostname errado.

Transição: “Podemos ainda escolher quais serviços auxiliares iniciar no desenvolvimento.”

E4-75 — Consulta | Profiles: serviços opcionais no Compose

Localização: PDF independente, página 76; apresentação principal, posição 182. **Tempo sugerido:** 1–3 min se houver uma dúvida pertinente.

O que o aluno deve levar: Profiles permitem escolher ferramentas ou tarefas opcionais sem duplicar toda a aplicação.

Fala de abertura sugerida: “Profiles permitem organizar serviços opcionais. Uma ferramenta administrativa pode ser útil em alguns momentos sem fazer parte da subida comum de app e db.”

Como conduzir: Leia apenas o serviço adicional ilustrado. Explique que o trecho precisa da configuração completa e não substitui o arquivo existente.

Explicação para desenvolver: O trecho ilustra um serviço adicional, não substitui o compose.yaml existente. App e db continuam sem profile para iniciar normalmente. A imagem e a porta são exemplos e precisam estar disponíveis se a equipe decidir experimentá-los; não há obrigação de baixar a ferramenta. Selecionar explicitamente um serviço na CLI também pode ativar esse serviço com profile. Profiles organizam combinações de serviços, mas não são autorização de acesso nem gestão de segredos.

Comando ou trecho de apoio: leia a intenção antes de executar. Trechos de configuração podem ser parciais; o arquivo completo do laboratório é a referência.

```
services:
  admin:
    image: adminer:4
    profiles: ["ferramentas"]
    ports:
      - "127.0.0.1:18082:8080"
```

```
# Na configuração completa com app e db:
# docker compose --profile ferramentas up -d
```

Transição: “Outra forma de adaptar o desenvolvimento é combinar configurações ou observar mudanças.”

E4-76 — Consulta | Overrides e Watch no desenvolvimento

Localização: PDF independente, página 77; apresentação principal, posição 183. **Tempo sugerido:** 1–3 min se houver uma dúvida pertinente.

O que o aluno deve levar: Confira a versão do Compose e a configuração resultante; conveniência local não dispensa testar a imagem final.

Fala de abertura sugerida: “Overrides combinam arquivos de configuração; Watch pode acompanhar mudanças conforme a versão e as regras definidas. Precisamos saber qual configuração efetiva será aplicada.”

Como conduzir: Mostre conceitualmente o uso de compose config para revisar a combinação. Diferencie sincronizar código de reconstruir dependências.

Explicação para desenvolver: Uma execução com arquivos explícitos pode usar docker compose -f compose.yaml -f compose.dev.yaml config para revisar a combinação antes de subir. As regras de merge variam por campo; não presuma que toda lista será substituída. Watch é recurso do plugin Compose e seu suporte depende da versão instalada. Uma mudança de dependência pode exigir rebuild, enquanto uma mudança de código pode usar sync se o processo recarrega. O objetivo é encurtar o ciclo de desenvolvimento sem tornar a configuração final um mistério.

Transição: “A construção também tem recursos próprios para cache e plataformas.”

E4-77 — Consulta | BuildKit, Buildx e cache de construção

Localização: PDF independente, página 78; apresentação principal, posição 184. **Tempo sugerido:** 1–3 min se houver uma dúvida pertinente.

O que o aluno deve levar: Otimizar a construção e controlar a plataforma complementam a reprodução; não substituem testes.

Fala de abertura sugerida: “BuildKit e Buildx ampliam possibilidades de construção. Eles não são o runtime da aplicação. O recurso adequado depende do problema: cache, segredo de build ou plataforma de destino.”

Como conduzir: Retome o cache simples antes de citar opções avançadas. Diga que multiplataforma precisa de compatibilidade efetiva da aplicação e da base.

Explicação para desenvolver: O cache de camadas explicado na trilha principal é o primeiro passo. Cache mounts podem reaproveitar downloads sem colocar esse cache no artefato final. Build secrets evitam fornecer credenciais por ARG ou ENV persistentes, mas ainda exigem que o comando não copie o segredo para a saída. Buildx e BuildKit oferecem capacidades de construção; não devem ser confundidos com o runtime que inicia a aplicação. Uma imagem multiplataforma exige que código, base e dependências funcionem nas plataformas publicadas.

Transição: “O último mapa ajuda a localizar outras capacidades do ecossistema.”

E4-78 — Consulta | Outros recursos do ecossistema

Localização: PDF independente, página 79; apresentação principal, posição 185. **Tempo sugerido:** 1–3 min se houver uma dúvida pertinente.

O que o aluno deve levar: A entrega desta aula usa os recursos necessários ao projeto; conhecer opções não cria novas exigências.

Fala de abertura sugerida: “Conhecer o ecossistema ajuda a formular perguntas melhores. Registry, context, análise de imagem e cluster têm responsabilidades distintas; não precisam entrar todos no projeto.”

Como conduzir: Responda somente à necessidade que motivou a consulta. Oriente a verificar documentação e versão antes de adotar outro recurso. Retome o objetivo de execução verificável.

Explicação para desenvolver: Faça uma passagem pelo mapa sem instalar componentes adicionais. Registry armazena imagens; não executa a aplicação. Context permite apontar comandos para outro Engine, portanto conhecer o destino importa. Uma análise de vulnerabilidades ajuda a priorizar correções, mas depende de dados atualizados e não comprova segurança total. Swarm introduz cluster e estado desejado de serviços, tema além da execução em um host. Algumas capacidades de produtos e planos mudam; consulte a documentação oficial antes de adotá-las.

Transição: “Finalize a dúvida e volte ao projeto ou ao encerramento; não abra outra trilha obrigatória.”

Apoio durante a aula: dúvidas e contingências

Respostas curtas para confusões frequentes

Se o aluno disser...	Como responder e retomar
"Container é uma VM pequena."	"Vamos olhar a estrutura: onde está o kernel?" Retome E4-15 e o compartilhamento no ambiente Linux.
"Node e containerd fazem a mesma coisa."	"Um executa o JavaScript; o outro participa da gestão da execução em containers." Retome E4-10.
"A imagem está rodando."	"A instância criada a partir dela está executando um processo." Retome E4-09 e localize a instância em ps.
"EXPOSE já abriu a porta."	"A publicação está em ports ou -p; EXPOSE é metadado." Retome E4-40.
"Vou usar localhost para falar com db."	"De dentro de qual processo sai essa conexão?" Trace app → db:5432 em E4-39.
"Healthy garante toda a aplicação."	"Qual comando produziu esse estado e o que ele testa?" Compare SELECT 1 com /visitas em E4-44.
"Restart vai corrigir unhealthy."	"A política reage à saída conforme suas regras; precisamos corrigir a causa do check." Retome E4-48.
"Tem volume, então tem backup."	"O teste mostrou reutilização local. Onde está a cópia recuperável e o teste de restauração?" Retome E4-51.
"A IA disse, então é a causa."	"Qual teste distingue essa hipótese de outra?" Use E4-59 e registre o resultado.

Se a demonstração não funcionar

Situação	Conduta do professor	Limite de tempo e continuidade
Engine não responde.	Conferir Docker ativo e destino/contexto usado. Dizer que a falha antecede a aplicação.	Até 2 min; usar saída do ensaio identificada e manter a explicação.
Download/build lento por rede.	Usar imagens e build preparados. Se não existirem, mostrar arquivo e saída prévia identificada.	Não ocupar o tempo da prática aguardando sem previsão.
Nome/porta já em uso.	Identificar a instância ou serviço e escolher recurso livre, atualizando os comandos seguintes.	Não remover serviço desconhecido para liberar a demonstração.
logs vazio no primeiro servidor.	Explicar que responder HTTP não implica emitir log. Mostrar a resposta e os processos.	É um resultado possível; não gastar tempo tentando "consertar logs".
API isolada dá ready 503.	Explicar que a dependência ainda não está integrada.	É o resultado esperado de E4-32.
up --wait falha na integração.	Parar a sequência; consultar ps -a/logs e localizar o check que falhou.	Até 3 min de diagnóstico compartilhado; depois contingência e registro.
Banco iniciou, mas ready ainda falha.	Aguardar recuperação da conexão da API e repetir ready; só então executar a operação.	Se não voltar, não declarar recuperação; registrar o sintoma real.

Situação	Conduta do professor	Limite de tempo e continuidade
Uma equipe está bloqueada.	Reduzir o escopo a uma operação verificável e uma hipótese de falha.	Ajudar a destravar sem substituir o projeto pela API do professor.

Frase de contingência: “Este é o estado que estamos observando agora. A expectativa era outra, então ainda não vou considerar o teste concluído. Vou mostrar um registro do ensaio, identificar a diferença e deixar a investigação registrada sem consumir o trabalho das equipes.”

Encerramento do ambiente demonstrativo

Na pasta Práticas/encontro-4-containers, o encerramento normal da stack é:

```
docker compose down
```

Isso preserva o volume nomeado. A instância e4-primeiro já deve ter sido parada/removida em E4-22, e a API isolada encerrada em E4-32. Não execute remoção global. Não misture os projetos E4, E5 e E6 ao usar terminais.

Checklist do professor ao concluir

- Os alunos distinguiram os dois runtimes, imagem e instância.
- Uma operação mostrou a integração, além de estado Up/healthy.
- A persistência foi explicada por uma comparação antes/depois.
- Uma falha foi ligada a hipótese, evidência e mudança específica.
- As quatro verificações tiveram respostas explicadas.
- As equipes registraram o incremento no próprio repositório, com instruções e evidências.
- Pendências ficaram identificadas; não houve declaração de teste que não ocorreu.
- A ponte para entrega controlada no E5 ficou clara.

Materiais de apoio e referências

Base do roteiro: apresentação independente “Encontro 4 - Containers e Compose”, 79 páginas; conteúdo e notas de conteudo.json; roteiro de quatro horas e READMEs do laboratório existentes no repositório. O texto foi ampliado para condução oral em 24/09/2026. Nenhum slide foi modificado.

Localização no repositório:

- Slides/Encontro 4 - Recursos/Encontro 4 - Containers e Compose.pdf
- Slides/Encontro 4 - Recursos/Guia ampliado do professor.md
- Práticas/encontro-4-containers/PRIMEIROS-PASSOS.md
- Práticas/encontro-4-containers/README.md

Referências oficiais para os pontos de rede, saúde, reinício e armazenamento (conferidas em 24/09/2026):

- [Rede no Compose](<https://docs.docker.com/compose/how-tos/networking/>) — referência para E4-39-40.
- [Ordem de inicialização no Compose](<https://docs.docker.com/compose/how-tos/startup-order/>) — referência para E4-45-46.
- [Políticas de reinício](<https://docs.docker.com/engine/containers/start-containers-automatically/>) — referência para E4-48.
- [Volumes](<https://docs.docker.com/engine/storage/volumes/>) — referência para E4-43 e 51.

As demais referências técnicas permanecem no guia original do encontro. Este roteiro organiza a condução da aula; os resultados descritos como esperados devem ser confrontados com a execução real.